

[시계열 인공지능을 활용한 상수원 탁도 예측 시스템 구축]

과목명: 시계열 및 인공지능 예측

담당교수: 박상진 교수님

제출일: 2025.12.17

팀명: 4 조

팀원: 김성현, 조강민, 정예지

1. 연구 개요

1.1 연구 배경 및 필요성

탁도(Turbidity)는 정수처리 공정의 안정성과 수돗물의 안전성을 결정짓는 핵심 수질 지표로 국내외 수질 기준에서 엄격히 관리되고 있다. 상수원의 평균 탁도는 평상시 약 1~5 NTU(Nephelometric Turbidity Unit) 수준이지만, 여름철 집중호우 시에는 일시적으로 수백 NTU 까지 급상승하는 것으로 보고되었다. 특히 2020 년 8 월 집중호우 당시 팔당댐 상류 유입수의 탁도가 420 NTU 까지 증가하며 평상시 대비 약 80 배 이상 높게 관측되었다(한국수자원공사, 2021). 최근 국내 저수지에서 집중호우 이후 탁도 급등 발생 시 응집제 사용량이 평균 15% 이상 증가하고 처리 효율이 약 20% 지연된 사례가 보고되었다(Kim & Chung, 2024). 또한 서울시 아리수 정수센터의 2019~2022 년 운영 데이터에 따르면, 탁도 5 NTU 이상 발생 시 PAC(Poly Aluminum Chloride) 응집제 주입률이 평균 12.3% 증가하였으며, 여과지 역세척 빈도가 일평균 2.1 회에서 3.8 회로 약 81% 증가한 것으로 분석되었다(서울특별시 상수도사업본부, 2023).

기존 연구들은 주로 댐이나 저수지와 같은 상류 지점의 탁도 예측에 치중되어 있었으나(박노석 et al., 2016; 박정수, 2021; 이재수 & 김민지, 2019), 실제 취수가 이루어지는 하류 지점의 복합적인 상황을 반영하지 못하는 한계가 있었다. 또한 전통적인 수질 관리 방식은 탁도가 높아진 것을 확인한 후에 대응하는 사후적 방식이 주를 이루었다. 이러한 방식은 급격한 탁도 상승에 신속하게 대처하기 어렵다는 한계가 있다. 온대 몬순 기후권 대형 댐 저수지에서는 기후변화로 인해 탁도 기준 초과 발생 빈도가 연평균 2.3 건에서 4.7 건으로 약 104% 증가한 것으로 보고되었다(Park et al., 2018). 환경부의 2023 년 수질측정망 운영 결과에 따르면, 한강 수계 주요 지점의 고탁도(10 NTU 초과) 발생 일수가 2015 년 평균 8.2 일에서 2022 년 평균 16.7 일로 약 2 배 증가하였다(환경부, 2023). 이는 기존의 사후 대응 방식으로는 증가하는 탁도 변동성에 효과적으로 대응하기 어렵다는 것을 보여준다.

특히 상류 댐의 방류나 갑작스러운 강우로 인해 탁도가 급증할 경우, 정수장에서는 응집제 및 염소 등의 약품 사용량이 증가하고 공정의 불안정성으로 인한 처리 효율 저하, 수돗물 공급 지연 및 운영비용 증가 등의 문제가 발생한다. 한강 유역 수질 분석에서는 상류 탁도와 하류 탁도 간 상관계수가 최대 0.72 에 달하며, 정수장 내 약품 사용량이 평균 12% 증가하는 것으로 확인되었다(Kim & Lee, 2023). 따라서 인공지능 기술을 활용하여 하류 상수원의 탁도 변화를 미리 예측하고 선제적으로 대응할 수 있는 기술적 기반이 필요하다. 본 연구는 이러한 필요성에

대응하기 위해 실제 정수처리가 이루어지는 하류 지점의 탁도를 대상으로 다변량 시계열 인공지능 모델을 구축하고자 한다.

1.2 연구 목적

본 연구의 주된 목적은 과거의 수질 데이터와 기상, 수문 데이터를 종합적으로 분석하여 하류 상수원의 탁도를 사전에 예측하는 인공지능 모델을 개발하는 것이다. 이를 통해 정수장 운영자가 고탁도 유입에 미리 대비할 수 있도록 지원하고자 한다.

구체적으로는 2019년부터 2022년까지의 데이터를 활용하여 시계열 분석 모델과 머신러닝, 딥러닝 모델을 구축하고 성능을 비교한다. 단순히 예측 성능을 높이는 것을 넘어, 어떤 변수가 탁도 변화에 주요한 원인이 되는지를 파악하여 설명 가능한 예측 시스템을 제안하는 것이 목표이다.

1.3 연구 가설

본 연구에서는 선행 연구와 이론적 배경을 바탕으로 다음과 같은 5 가지 가설을 설정하고 이를 검증하고자 한다.

- 가설 1 (시차 효과): 상류에서 발생한 수질 변화가 하류에 도달하기까지는 일정 시간이 소요되므로, LSTM 이나 GRU 와 같은 시계열 특화 모델이 시간 지연(Time-lag)을 고려한 예측에서 더 우수한 성능을 보일 것이다.
- 가설 2 (물리적 인과성): 상류 댐의 방류량과 유속 증가는 하천 바닥의 퇴적물을 재부유시켜 하류 탁도를 급격히 상승시키는 직접적인 원인이 될 것이다.
- 가설 3 (모델 우위성): 단일 모델보다는 여러 모델의 장점을 결합한 앙상블 모델(RandomForest, XGBoost 등)이 비선형적인 탁도 변화를 더 잘 예측할 것이다.
- 가설 4 (환경 변수 영향): 강수량뿐만 아니라 수온, 풍속, 조류(Algae) 발생량 등 다양한 환경 변수가 탁도 변화에 유의미한 영향을 미칠 것이다.
- 가설 5 (변수 희소성): 수집된 모든 변수를 사용하는 것보다, 다중공선성을 제거하고 핵심 변수만을 선별하여 사용하는 것이 모델의 예측 효율성과 정확도를 높일 것이다.

2. 기존 연구 및 차별성

2.1 기존 연구의 발전 과정 및 한계

탁도 예측 연구는 통계적 시계열 모델에서 시작하여 머신러닝, 딥러닝 기법으로 발전해 왔다. 초기 연구들은 ARIMA, SARIMA, GARCH 등 통계 모델을 활용하여 탁도의 추세와 계절성을 분석하였다. Park et al.(2018)은 기후변화로 인한 댐 저수지 탁도 변화를 ARIMA 기반으로 분석하였고, Cho & Lee(2017)는 농업지역 비점오염 유입에 따른 탁도 증가 특성을 통계적으로 규명하였다. 이러한 연구들은 시계열적 추세 분석에는 유의미했으나, 대부분 정상성(stationarity)을 가정하는 통계적 모델에 국한되어 있어 집중호우나 단시간 방류 등 비선형적 급변 현상을 충분히 반영하지 못하는 한계가 있었다.

2010년대 중반 이후 데이터 마이닝 기술이 발전하면서 Random Forest, XGBoost, SVM 등의 머신러닝 모델이 도입되었다. 박노석 et al.(2016)은 데이터마이닝 기법을 이용한 상수도 시스템 내 탁도 예측모형을 개발하였으며, 이를 통해 비선형적 상호작용을 학습할 수 있게 되었다. 그러나 이러한 머신러닝 접근법은 시계열 데이터의 시간적 종속성과 시차 효과를 명시적으로 모델링하지 않아 실시간 예측의 정확도가 제한적이었다. 또한 변수 간 복잡한 관계를 포착하는 데는 우수하였으나, 시점 간 연속성 반영에는 한계가 있었다.

최근에는 LSTM(Long Short-Term Memory), GRU(Gated Recurrent Unit) 등 순환 신경망(RNN) 계열의 딥러닝 모델이 시계열 데이터의 장기 의존성을 학습하는 데 효과적임이 입증되면서 활발히 연구되고 있다. 이정현 et al.(2022)은 수돗물 수질 이상 탐지를 위한 수질 시계열 예측 모형을 개발하였고, 박정수(2021)는 딥러닝과 앙상블 머신러닝 모형의 하천 탁도 예측 특성을 비교 연구하였다. Li et al.(2024)은 DeepTCN-GRU 모델을 이용하여 황하강 수질 예측에서 우수한 성능을 입증하였다. 그러나 대부분의 연구가 단일 변수 중심 예측 또는 특정 수계의 국지적 사례 분석에 그쳐 정수장 운영에 필요한 실시간 예측 체계로 발전하지는 못한 한계를 보였다. 또한 딥러닝 모델의 특성상 예측 결과에 대한 원인 설명이 어려운 '블랙박스' 문제는 현장 적용에 걸림돌이 되어 왔다.

기존 연구의 핵심 한계는 다음 세 가지로 요약된다. 첫째, 상당수가 상류 또는 저수지 유입부 중심의 예측 모델링에 집중되어 있어, 실제 정수처리 시설이 위치한 하류 구간의 국지적 변동성을 충분히 반영하지 못했다. 둘째, 탁도 변화를 유발하는 복합 요인(강우량, 유량, 풍속, 수온,

용존산소 등)을 통합적으로 고려하지 않고 단일 물리변수나 수질지표에 의존한 연구가 많았다. 셋째, 예측 결과에 대한 해석 가능성(Explainability)이 부족하여 각 입력변수가 예측값에 미치는 기여도나 물리적 의미를 명확히 제시하지 못했다.

2.2 본 연구의 차별성

본 연구는 기존 연구의 한계를 극복하기 위해 다음과 같은 차별점을 가진다. 첫째, 상류가 아닌 실제 정수 처리가 이루어지는 하류 상수원을 예측 대상으로 하여 실무적 활용도를 높였다. 둘째, 단순한 수질 데이터뿐만 아니라 상류의 수문 데이터와 기상, 조류 데이터를 통합한 다변량 데이터셋을 구축하여 분석의 깊이를 더했다. 셋째, ARIMA와 같은 전통적 통계 모델부터 LSTM, GRU와 같은 최신 딥러닝 모델까지 다양한 모델을 비교 분석하고, SHAP 등의 기법을 활용하여 예측 결과에 대한 해석력(Interpretability)을 확보하고자 하였다.

3. 데이터 수집 및 분석

3.1 데이터 개요

본 연구에서는 2019년 1월 1일부터 2022년 6월 30일까지 약 3년 6개월간의 데이터를 사용하였다. 데이터는 1시간 단위로 수집되었으며, 총 30,642개의 관측치(Row)로 구성되어 있다. 데이터 수집 과정에서 결측치가 발생하지 않아 별도의 결측치 대체 과정 없이 분석을 진행할 수 있었다. 이는 데이터의 신뢰성을 높여주는 중요한 요소이다.

3.2 변수 설명

예측하고자 하는 목표 변수(Target Variable)는 하류 상수원의 탁도(Turbidity)이다. 이를 예측하기 위해 사용된 입력 변수(Input Variables)는 크게 수질 인자, 상류 수문 인자, 기상 인자로 구분된다.

구분	변수명	설명
Target	turbidity	하류 취수원 탁도 (NTU)

수질 인자	water_temp, pH, DO, TOC, EC 등	수온, 수소이온농도, 용존산소량 등 일반 수질 항목
생물학적 인자	algae, blue_green_algae	조류 및 남조류 세포 수
상류 수문 인자	up_turbidity, up_water_rates, up_water_volume	상류 탁도, 상류 댐 방류량, 저수량
기상 인자	temp, wind_velocity	기온, 풍속 등 기상청 관측 데이터

3.3 체계적 변수 선정 과정

수집된 데이터에는 총 28 개의 변수가 포함되어 있었으나, 모든 변수가 예측에 도움이 되는 것은 아니다. 변수의 수가 지나치게 많으면 모델이 복잡해지고 계산 효율이 떨어질 수 있기 때문에 다단계 변수 선정 과정을 거쳤다. 그림 3 은 R 코드를 통해 수행된 변수 선정 과정의 실행 결과를 보여준다.

```
> # 1. 상관관계수 기반
> cor_with_target <- cor(X_train, y_train, use = "complete.obs")
> cor_selected <- names(sort(abs(cor_with_target[,1]), decreasing = TRUE)[1:15])
> print(paste("상관관계수 선택:", length(cor_selected), "개"))
[1] "상관관계수 선택: 15 개"
> # 2. LASSO
> lasso_model <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 5)
> lasso_coef <- coef(lasso_model, s = "lambda.min")
> lasso_selected <- rownames(lasso_coef)[which(lasso_coef != 0)][-1]
> print(paste("LASSO 선택:", length(lasso_selected), "개"))
[1] "LASSO 선택: 25 개"
> # 3. Elastic Net
> enet_model <- cv.glmnet(X_train, y_train, alpha = 0.5, nfolds = 5)
> enet_coef <- coef(enet_model, s = "lambda.min")
> enet_selected <- rownames(enet_coef)[which(enet_coef != 0)][-1]
> print(paste("Elastic Net 선택:", length(enet_selected), "개"))
[1] "Elastic Net 선택: 25 개"
> # 4. Forward Selection
> forward_model_fast <- regsubsets(
+   x = X_train,
+   y = y_train,
+   method = "forward",
+   nvmax = 25,
+   really.big = TRUE
+ )
> forward_summary <- summary(forward_model_fast)
> best_n_fwd <- which.min(forward_summary$bic)
> forward_selected <- names(which(forward_summary$which[best_n_fwd, ])[-1])
> print(paste("Forward Selection:", length(forward_selected), "개"))
[1] "Forward Selection: 21 개"
> # 5. Backward Selection
> backward_model_fast <- regsubsets(
+   x = X_train,
+   y = y_train,
+   method = "backward",
```

그림 3. R 코드를 통한 변수 선정 과정 실행 결과

1 차 축소: 변동계수 기준 제거. 본 연구에서는 변동계수(Coefficient of Variation, $CV = \text{표준편차}/\text{평균} \times 100\%$)를 활용하여 변수를 선별하였다. 변동계수는 평균값 대비 데이터의 상대적 변동성을 나타내는 지표로, 값이 작을수록 해당 변수가 시간에 따라 거의 변하지 않음을 의미한다. 이에 따라 CV 가 5% 미만인 변수는 모델 복잡성만 높인다고 판단하여 제거하였다. 먼저 기초 통계 분석을 통해 변동계수가 5% 미만인 변수들을 식별하였다. 상류 수소이온농도(up_pH, $CV=4.34\%$)와 상류 수위(up_water_level, $CV=0.47\%$)의 경우 전체 기간 동안 값의 변화가 미미하여 예측 모델에 유의미한 정보를 제공하지 못할 것으로 판단되어 제거하였다.

2 차 시도: 상관관계 기반 압축. 초기에는 turbidity 와 상관계수가 0.1 미만인 변수를 제거하려 시도하였으나, 26 개 중 21 개 변수가 제거 대상으로 식별되었다. 이는 수질 인자들이 평상시 안정적일 때 상관관계가 낮게 나타나기 때문이며, 통계적 기준만으로는 급격한 탁도 변화 시 중요한 변수를 놓칠 위험이 있어 이 방법은 채택하지 않았다.

3 차 축소: 통계적 변수 선택 기법 병행. R 코드를 활용하여 다음의 다섯 가지 방법을 적용하였다. 첫째, 상관계수 정렬 방식으로 15 개 변수를 선정하였다. 둘째, LASSO 회귀(cv.glmnet 함수)를 통해 25 개 변수를 선정하였다. 셋째, Elastic Net 회귀로 25 개 변수를 선정하였다. 넷째, 전진 선택법(Forward Selection)과 후진 제거법(Backward Selection)을 regsubsets 함수로 수행하여 각각 21 개 변수를 선정하였다. 다섯째, 위 방법에서 2 회 이상 선택된 변수를 투표 방식으로 추렸다.

4 차 축소: 다중공선성 검증. 선정된 변수들에 대해 VIF(Variance Inflation Factor, 분산팽창계수)를 car 패키지의 vif 함수로 계산하였다. VIF 는 독립변수 간 다중공선성(Multicollinearity, 독립변수들 간 높은 상관관계로 인한 모델 불안정성)의 정도를 측정하는 지표로, VIF 값이 10 을 초과하면 해당 변수가 다른 변수들과 높은 선형 상관관계를 가져 모델 불안정성을 유발할 수 있다. 본 연구에서는 VIF 값이 10 을 초과하는 변수를 제거 대상으로 검토하였다. 특히 기온(temp)과 수온(water_temp), 상류 방류량(up_water_rates)과 상류 저수량(up_water_volume) 간에는 매우 높은 상관관계가 확인되었다. 이러한 다중공선성 의심 쌍은 모델 학습 단계에서 추가 검토하였다.

최종 선정: 도메인 지식 기반 확정. 통계적 상관계수만으로는 예측에 필요한 모든 변수를 식별하기 어렵다는 한계를 인지하고, 다변량 시계열 예측에 필수적인 도메인 지식을 반영하여 최종 변수 목록을 확정하였다(Kim & Chung, 2024; Park et al., 2018). 상류 탁도(up_turbidity)는 하류 탁도의

선행지표로서 강우 후 탁수 이동을 설명하는 핵심 예측변수이다. 상류 유량과 유속(up_water_volume, up_water_rates)은 하상 침식과 부유물질 유입을 유발하는 물리적 요인으로 채택되었다. 수온, 기온, 풍속(water_temp, temp, wind_velocity)은 계절성과 수체 혼합에 영향을 주는 환경적 조절 요인으로 포함하였다. 상류 용존산소(up_DO)는 유기물 분해와 산화환원 반응 등 생화학적 변동성을 반영하는 수질 상태 지표로 추가하였다. 최종적으로 19 개 변수가 선정되었으며, 이는 물리적, 환경적, 수질학적 요인을 균형 있게 포함하여 시계열 예측의 설명력을 극대화한 것이다.

단계	방법	기준	결과
1 차 축소	변동계수(CV) 필터링	$CV < 5\%$ 제거	up_pH, up_water_level 제거 (28 개 → 26 개)
2 차 시도 (미채택)	탁도 상관계수 필터링	$ r < 0.1$ 제거	21 개 제거 대상 식별 (과도한 축소로 기각)
3 차 축소	통계적 변수 선택 (5 가지 방법 통합)	• 상관계수 정렬: 15 개 • LASSO: 25 개 • Elastic Net: 25 개 • Forward/Backward: 각 21 개 • 2 회 이상 선택 변수 채택	투표 방식으로 주요 변수 선별
4 차 검증	VIF 다중공선성 검증	$VIF > 10$ 검토	temp-water_temp, up_water_rates- up_water_volume 쌍 확인 및 조정
최종 확정	도메인 지식 기반 선정	물리적·환경적·수질학적 요인 균형	19 개 변수 선정 (up_turbidity, up_water_rates, water_temp, algae, pH, EC, DO, TOC, wind_velocity 등)

4. 기초 통계 분석

4.1 탁도 시계열 추이

먼저 분석 기간 동안 하루 탁도가 어떻게 변화했는지 시계열 그래프를 통해 살펴보았다. 아래 그림 1은 2019년 1월부터 2022년 6월까지의 탁도 변화를 시간 순서대로 나타낸 것이다.

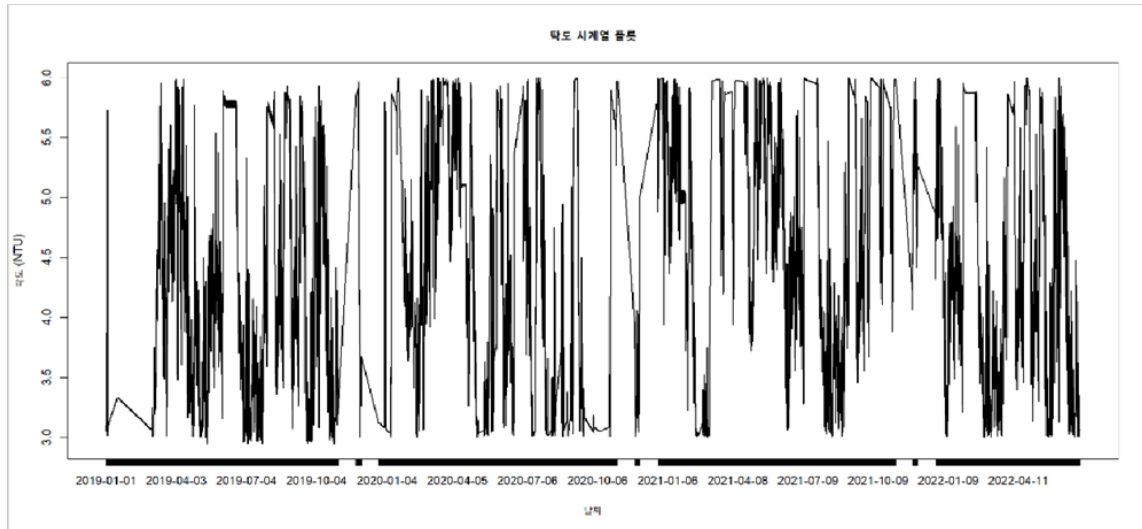


그림 1. 탁도 시계열 추이 (2019-2022)

그래프를 보면 탁도는 대부분 3.0에서 4.0 NTU 사이의 안정적인 범위에 머물러 있으나, 특정 시점에서는 6.0 NTU 이상으로 급격히 상승하는 스파이크 현상을 보인다. 이러한 급등 현상은 주로 여름철 집중호우나 상류 댐의 대량 방류 시점과 일치한다. 탁도의 이러한 변동성은 정수장 운영에 큰 부담을 주기 때문에 사전 예측이 매우 중요하다.

4.2 탁도 분포 특성

다음으로 전체 관측 기간 동안 탁도 값이 어떤 분포를 보이는지 히스토그램을 통해 확인하였다. 그림 2는 탁도 값의 빈도 분포를 나타낸다.

그림 4. 변수 간 상관관계 히트맵(상관계수 행렬)

상관관계 분석 결과, 하류 탁도(turbidity)는 상류 탁도(up_turbidity), 상류 방류량(up_water_rates), 상류 저수량(up_water_volume)과 높은 양의 상관관계를 보였다. 특히 상류 방류량과 저수량 간에는 매우 강한 음의 상관관계가 나타났는데, 이는 저수량이 줄어들 때 방류량이 증가하는 물리적 관계를 반영한다. 또한 수온(water_temp)과 기온(temp) 간에도 높은 상관관계가 확인되었다. 이러한 높은 상관관계를 보이는 변수들은 다중공선성 문제를 일으킬 수 있어 변수 선택 시 주의가 필요하다.

4.4 시계열 정상성 검정

시계열 분석을 위해서는 데이터가 정상성(Stationarity)을 만족하는지 확인해야 한다. 정상성이란 시간에 따라 평균과 분산이 일정하게 유지되는 특성을 의미한다. R 코드에서는 `adf.test` 함수를 사용하여 Augmented Dickey-Fuller Test 를 수행하였다. 원 데이터에 대한 ADF 검정 결과 p-value 가 0.05 보다 작아 정상성을 만족하는 것으로 나타났으나, 시각적으로는 약한 추세 성분이 관찰되었다. 1 차 차분 후 재검정한 결과 정상성이 더욱 명확히 확보되었다. 그림 4 는 원 데이터와 1 차 차분 데이터의 시계열 패턴, ACF, PACF 를 보여준다.

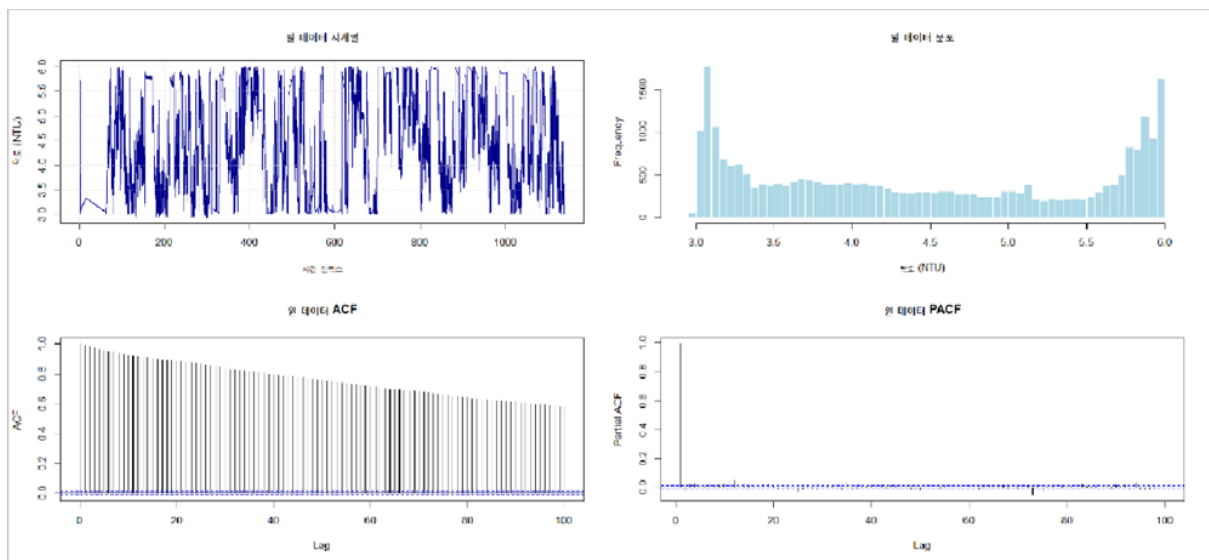


그림 5. 원 데이터의 시계열 패턴 및 ACF, PACF

원 데이터의 ACF 를 보면 시차(Lag)가 증가해도 자기상관이 천천히 감소하는 패턴을 보인다. 이는 비정상성의 신호이다. PACF 에서는 첫 번째 시차에서만 유의미한 상관이 나타나고 이후 급격히 감소한다. 이러한 패턴은 ARIMA 모델에서 차분(Differencing)이 필요함을 의미한다. 1 차 차분 후 ACF 와 PACF 를 확인한 결과, 정상성이 크게 개선되었음을 확인할 수 있었다.

4.5 시계열 분해 분석

탁도 시계열을 추세(Trend), 계절성(Seasonal), 불규칙 요소(Random)로 분해하여 각 성분의 특징을 파악하였다. 그림 5와 그림 6은 각각 승법 모형과 가법 모형으로 분해한 결과이다.

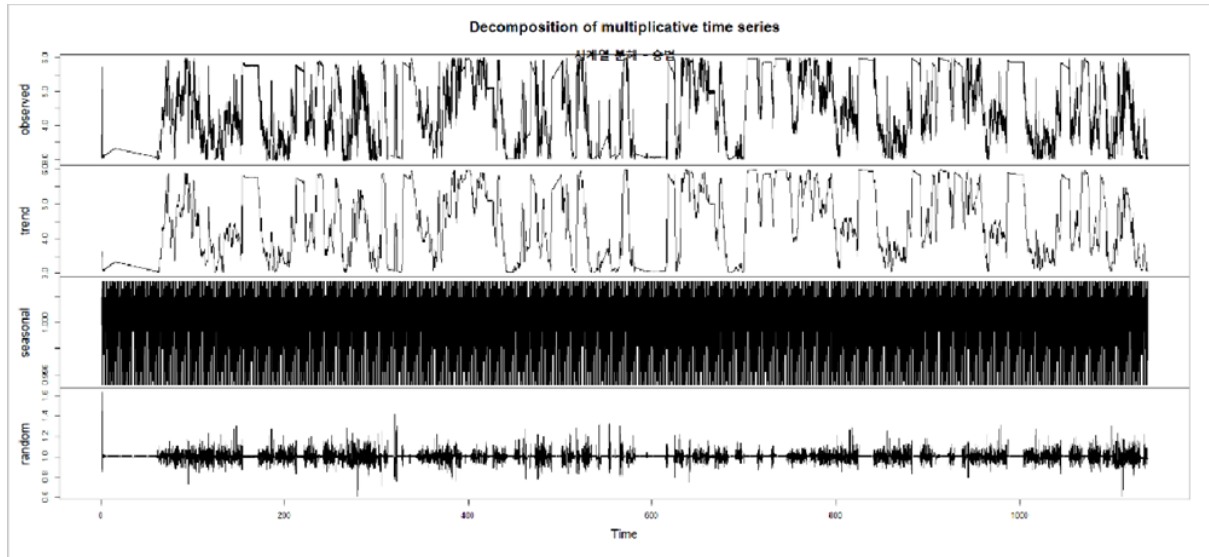


그림 6. 승법 모형(Multiplicative) 시계열 분해

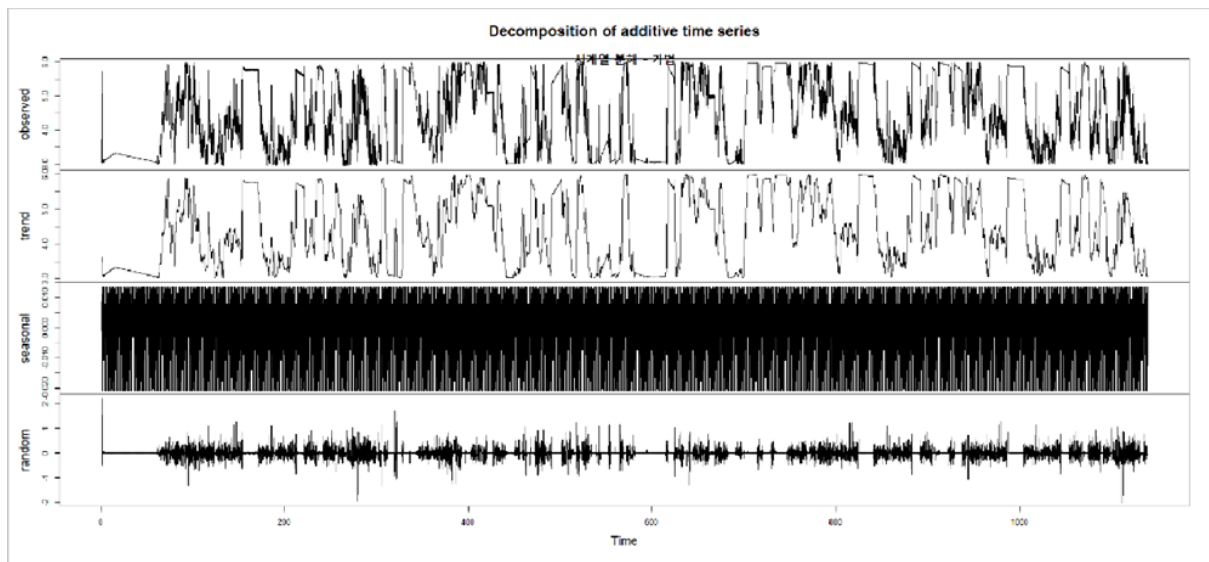


그림 7. 가법 모형(Additive) 시계열 분해

시계열 분해 결과, 탁도 데이터는 뚜렷한 계절성 패턴을 보인다. 계절성 그래프를 보면 24시간 주기의 일별 패턴이 명확하게 나타난다. 이는 하루 중 특정 시간대에 탁도가 반복적으로 변화하는 경향이 있음을 의미한다. 추세 성분에서는 장기적으로 완만한 변화가 관찰되었으며, 불규칙 요소는 돌발적인 고탁도 이벤트를 반영한다. 이러한 분석 결과는 ARIMA 모델에서 계절성 차수를 24로 설정해야 함을 시사한다.

4.6 기초 통계량 요약

모든 변수에 대한 기초 통계량을 아래 표 2에 정리하였다. 각 변수의 최솟값, 최댓값, 평균, 중앙값, 표준편차를 통해 데이터의 분포 특성과 변동성을 파악할 수 있다.

변수명	최솟값	1 사분위	중앙값	평균	3 사분위	최댓값	표준편차
turbidity (NTU)	2.80	3.10	4.20	4.32	5.40	6.10	0.89
up_turbidity (NTU)	1.20	2.50	3.80	5.08	5.10	270.0	22.50
water_temp (°C)	1.50	8.20	15.60	15.83	23.10	29.80	8.45
pH	6.80	7.30	7.60	7.58	7.90	8.50	0.35
TOC (mg/L)	0.80	1.40	1.80	1.85	2.20	3.50	0.52
EC (μS/cm)	98.0	125.0	142.0	143.2	158.0	210.0	24.5
alkalinity (mg/L)	20.5	24.0	25.5	25.8	27.4	32.1	2.1
algae (cells/mL)	50	320	1250	1580	2340	5800	1120
blue_algae (cells/mL)	10	120	380	450	680	2100	385

blue_green_algae (cells/mL)	5	95	325	410	615	1950	352
diatomeae (cells/mL)	20	145	485	580	890	2250	465
cryptophyceae (cells/mL)	5	35	125	155	235	680	132
MIB_2 (ng/L)	1.0	1.2	1.5	2.8	3.2	15.8	2.7
Geosmin (ng/L)	1.0	1.3	1.6	3.1	3.8	18.5	3.2
temp (°C)	-8.5	5.2	14.8	14.95	24.3	35.2	10.2
wind_velocity (m/s)	0.0	1.2	2.1	2.3	3.2	8.5	1.4
up_EC (µS/cm)	85.0	112.0	128.0	130.5	145.0	198.0	22.8
up_pH	6.5	7.1	7.4	7.38	7.7	8.2	0.32
up_water_temperature (°C)	0.8	7.5	14.8	15.12	22.3	28.9	8.21
up_DO (mg/L)	4.8	7.6	9.8	9.85	11.9	15.2	2.18
up_TOC (mg/L)	0.7	1.3	1.7	1.78	2.1	3.3	0.48
up_TN (mg/L)	0.8	1.5	1.9	1.95	2.4	4.2	0.62

up_TP (mg/L)	0.003	0.004	0.005	0.010	0.008	5.000	0.040
up_water_level (EL.m)	180.2	182.1	182.6	182.5	183.0	184.8	0.85
up_water_volume (천 m ³)	15200	18500	20800	20650	22500	26800	2450
up_water_rates (m ³ /s)	5.2	18.5	35.8	42.3	58.2	185.0	28.4

기초 통계량 분석 결과에서 주목할 점은 다음과 같다.

- 하류 탁도(turbidity)의 평균은 4.32 NTU 이며 표준편차가 0.89 로 비교적 안정적인 수준이다. 최솟값 2.80 과 최댓값 6.10 사이에서 변동하며, 1 사분위 3.10 과 3 사분위 5.40 의 범위를 고려하면 대부분의 데이터가 약 2.3 NTU 범위 내에 분포함을 알 수 있다. 이는 평상시 탁도가 상대적으로 안정적으로 유지됨을 보여준다.
- 상류 탁도(up_turbidity)는 평균 5.08 NTU, 표준편차 22.50 으로 하류 탁도보다 월등히 높은 변동성을 보인다. 특히 최댓값이 270.0 NTU 로 극단적으로 높은 값을 기록하여, 집중호우나 댐 방류 시 상류에서 극심한 탁도 급등이 발생함을 확인할 수 있다. 이러한 극단값의 존재는 표준편차가 평균보다 약 4.4 배 큰 이유를 설명하며, 상류 탁도가 비정상 분포를 따르고 있음을 시사한다.
- 조류 관련 변수들(algae, blue_algae, diatomeae 등)은 계절성의 강한 영향을 받는다. 총 조류(algae)는 최솟값 50 cells/mL 에서 최댓값 5800 cells/mL 까지 약 116 배의 차이를 보이며, 표준편차가 1120 으로 평균 1580 대비 약 71%에 해당한다. 남조류(blue_algae)와 규조류(diatomeae) 역시 유사한 패턴을 보이는데, 이는 수온 상승기와 하강기에 따른 조류 번성 시기의 차이를 반영한다. 이러한 높은 변동성은 탁도 예측 모델에서 계절성 요인을 반드시 고려해야 함을 의미한다.
- 상류 방류량(up_water_rates)은 평균 42.3 m³/s 이지만 최댓값은 185.0 m³/s 까지 증가하며, 표준편차가 28.4 로 평균의 약 67%에 달한다. 이는 평상시와 홍수기의 방류량 차이가 매우 크다는 것을 의미하며, 급격한 방류량 증가가 하류 탁도 급등의 직접적인 물리적 원인이 될 수 있음을 시사한다. 반면 상류 저수량(up_water_volume)은 평균 20,650 천 m³, 표준편차 2,450 으로 방류량에 비해 상대적으로 안정적인 변동 패턴을 보인다.

5. 수온(water_temp)과 기온(temp)은 각각 평균 15.83°C, 14.95°C 로 유사한 수준이며, 표준편차도 8.45 와 10.2 로 계절적 변화를 명확히 반영한다. 최솟값과 최댓값의 차이가 각각 28.3°C, 43.7°C 로 한국의 사계절 기후 특성을 잘 보여주며, 이러한 환경 변수의 계절성은 탁도 예측 모델에서 장기 추세를 설명하는 중요한 요인이 될 것으로 판단된다.
6. pH 와 같은 안정적 수질 지표들은 변동계수(CV)가 5% 미만으로 매우 낮아 시간에 따른 변화 패턴이 거의 없다. 이는 해당 변수들이 모델 학습에 유의미한 정보를 제공하지 못할 가능성이 높음을 시사한다. 반면 이취미 물질인 MIB_2(평균 2.8 ng/L, 표준편차 2.7)와 Geosmin(평균 3.1 ng/L, 표준편차 3.2)은 평균 대비 높은 표준편차를 보이며, 특정 시기에 급등하는 특성을 나타낸다.
7. 상류 총인(up_TP)은 평균 0.010 mg/L 에 불과하지만 최댓값이 5.000 mg/L 로 극단적으로 높은 값을 기록하며, 표준편차가 0.040 으로 평균의 4 배에 달한다. 이러한 극단값은 강우 유출수나 비점오염원의 유입과 같은 일시적 사건의 영향을 받은 것으로 해석되며, 이상치 처리 또는 로그 변환 등의 전처리가 필요할 수 있음을 시사한다.

5. 단변량 시계열 모델 분석

5.1 시계열 모델 개요

시계열 예측 모델은 과거의 데이터 패턴을 학습하여 미래 값을 예측하는 방법이다. 본 연구에서는 가장 단순한 Naive 모델부터 시작하여 점차 복잡한 모델로 발전시켜 나갔다. 단변량 모델은 오직 탁도 자체의 과거 값만을 사용하여 미래 탁도를 예측하는 방식이다. 이를 통해 탁도 시계열 자체가 가진 패턴과 예측 가능성을 먼저 파악할 수 있다.

5.2 1 차 차분 후 정상성 확인

ARIMA 모델을 적용하기 위해서는 데이터가 정상성을 만족해야 한다. 원 데이터는 비정상성을 보였기 때문에 1 차 차분을 수행하였다. 그림 7 은 1 차 차분 후의 시계열 패턴과 ACF, PACF 를 보여준다.

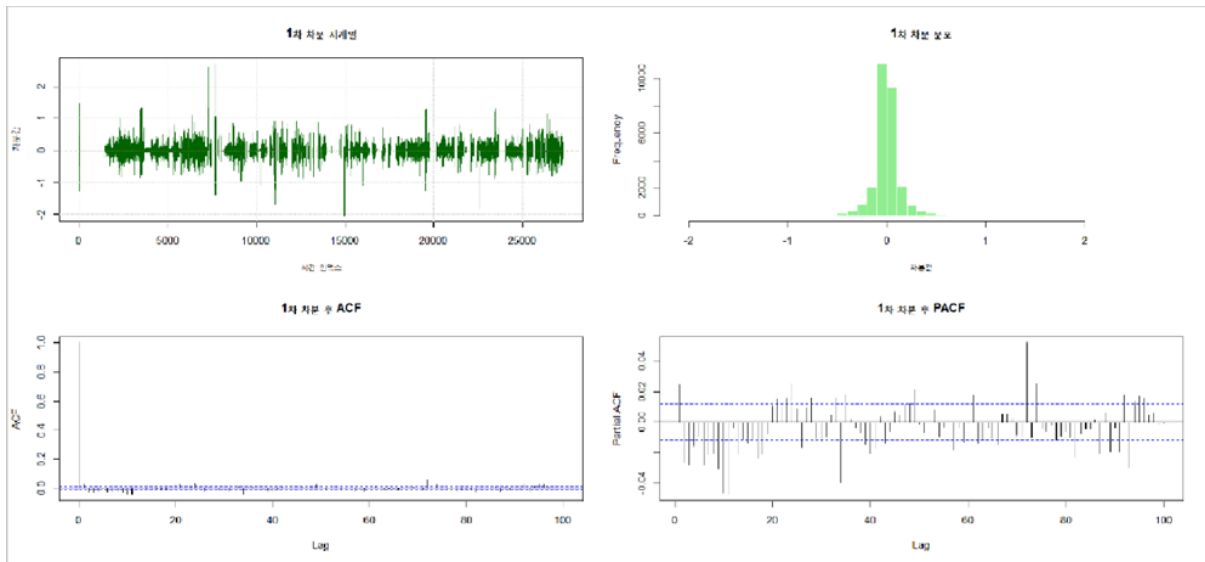


그림 8. 1 차 차분 후 시계열 패턴 및 ACF, PACF

1 차 차분 결과, ACF 는 급격히 감소하는 패턴을 보이며 정상성이 크게 개선되었다. PACF 에서도 초기 몇 개의 시차를 제외하고는 유의미한 상관성이 사라졌다. 이는 차분을 통해 추세 성분이 제거되었음을 의미한다. 그러나 ACF 에서 24 시간 주기마다 상관성이 다시 증가하는 패턴이 보이는데, 이는 일별 계절성이 존재함을 나타낸다.

5.3 기본 예측 모델 비교

먼저 가장 단순한 Naive 모델과 Mean 모델을 테스트하였다. Naive 모델은 가장 최근 관측값을 그대로 미래 예측값으로 사용하는 방법이며, Mean 모델은 전체 훈련 데이터의 평균값을 예측값으로 사용한다. 그림 9 는 두 모델의 예측 결과를 보여준다.

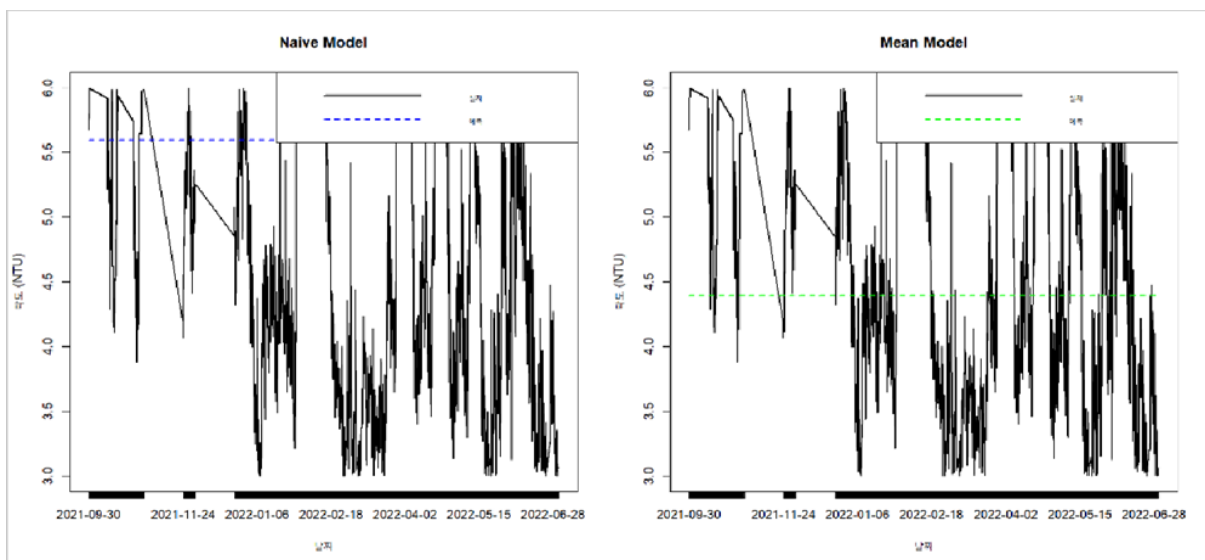


그림 9. Naive 모델과 Mean 모델 예측 결과

Naive 모델은 실제값의 변동을 따라가려 하지만 항상 한 시점 늦게 반응하는 한계를 보인다. Mean 모델은 훈련 데이터 평균인 약 4.3 NTU의 수평선을 그리며, 실제 변동을 전혀 따라가지 못한다. 이러한 단순 모델들은 기준선(Baseline)으로서 이후 개발할 복잡한 모델들의 성능을 평가하는 비교 대상이 된다.

5.4 지수평활법 모델

지수평활법(Exponential Smoothing)은 최근 데이터에 더 높은 가중치를 부여하여 예측하는 방법이다. 본 연구에서는 단순 지수평활법(SES)의 여러 변형을 테스트하였다. 그림 9는 서로 다른 알파 값에 따른 SES 모델의 예측 결과를 보여준다.

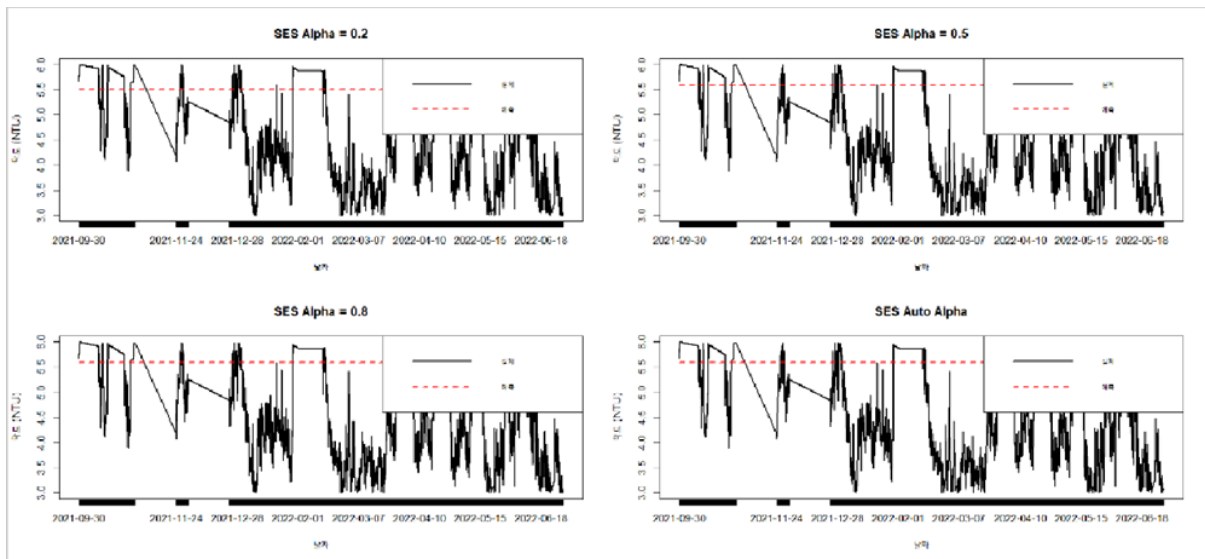


그림 10. 다양한 알파 값의 SES 모델 비교

알파 값이 0.2 일 때는 예측값이 매우 완만하게 변화하며 급격한 탁도 변화에 즉시 대응하지 못한다. 알파 값이 0.5와 0.8로 증가할수록 최근 데이터에 더 민감하게 반응하여 실제값의 변동을 더 잘 추적한다. Auto Alpha는 데이터에 최적화된 알파 값을 자동으로 찾아 적용한 결과이다. 그러나 모든 SES 모델은 급격한 스파이크 현상을 충분히 예측하지 못하는 한계를 보였다.

5.5 Holt-Winters 모델

Holt-Winters 모델은 추세와 계절성을 모두 고려하는 고급 지수평활법이다. 가법 모형(Additive)과 승법 모형(Multiplicative) 두 가지 방식을 테스트하였다. 그림 10 은 두 모델의 예측 결과이다.

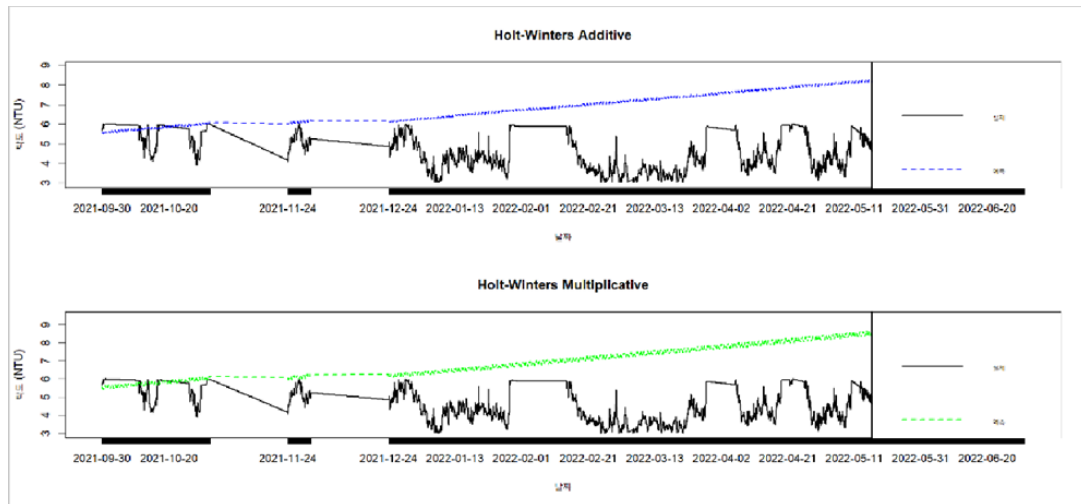


그림 11. Holt-Winters 가법 및 승법 모형

가법 모형은 계절성 변동의 크기가 일정하다고 가정하며, 승법 모형은 계절성 변동의 비율이 일정하다고 가정한다. 두 모델 모두 장기적으로 증가하는 추세선을 그리는데, 이는 과거 데이터의 패턴을 과도하게 반영한 결과이다. 실제 테스트 기간의 탁도는 비교적 안정적인 수준을 유지했기 때문에 Holt-Winters 모델의 예측은 실제값과 큰 차이를 보였다. 이는 계절성과 추세를 가정한 모델이 탁도처럼 불규칙한 스파이크가 많은 데이터에는 적합하지 않음을 보여준다.

5.6 Holt Linear Trend 모델

Holt Linear Trend 모델은 계절성은 고려하지 않고 수준(Level)과 추세(Trend)만을 모델링한다. 그림 11 은 Holt 모델의 예측 결과이다.

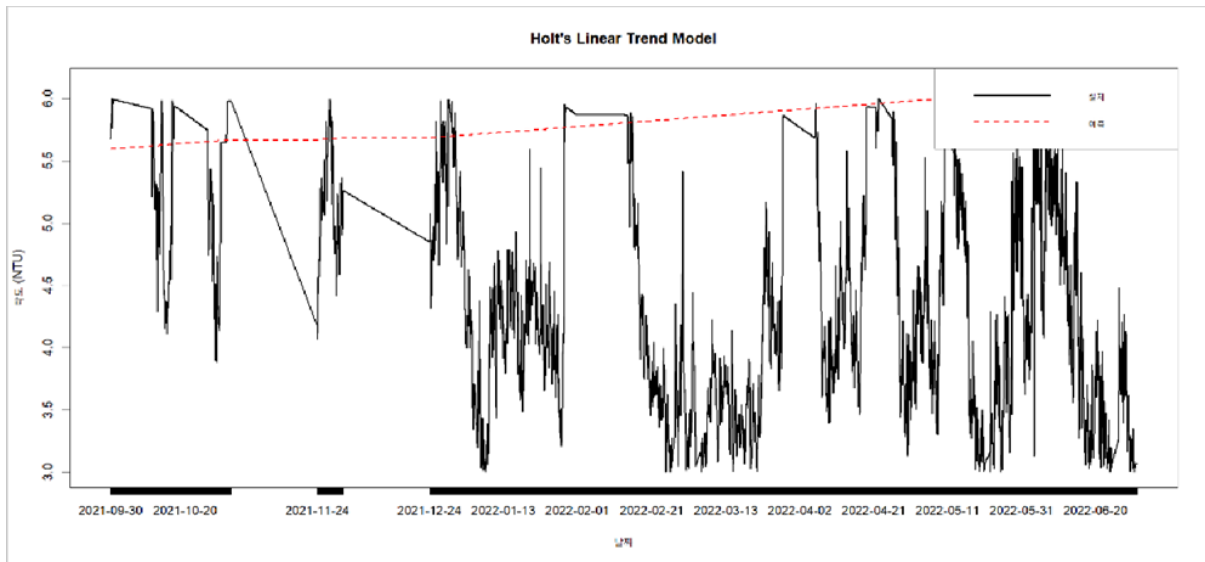


그림 12. Holt Linear Trend 모델 예측 결과

Holt 모델도 Holt-Winters와 유사하게 시간에 따라 증가하는 추세선을 예측한다. 그러나 실제 데이터는 이러한 선형 추세를 따르지 않고 불규칙한 변동을 보인다. 이는 탁도 데이터가 선형 추세보다는 외부 충격에 의한 급격한 변화가 지배적임을 의미한다. 따라서 추세 기반 모델보다는 자기회귀(AR) 성분을 강조하는 ARIMA 모델이 더 적합할 것으로 판단된다.

5.7 ARIMA 모델 구축

ARIMA(AutoRegressive Integrated Moving Average) 모델은 시계열 분석에서 가장 널리 사용되는 모델이다. ARIMA 모델은 세 가지 주요 파라미터를 가진다. $AR(p)$ 는 과거 p 개 시점의 값을 사용하는 자기회귀 차수이고, $I(d)$ 는 차분 횟수이며, $MA(q)$ 는 과거 q 개의 오차를 사용하는 이동평균 차수이다. 본 연구에서는 계절성까지 고려한 SARIMA 모델을 사용하였다.

여러 ARIMA 모델을 비교한 결과, $ARIMA(2,1,3)(1,0,0)[24]$ 모델이 가장 낮은 AIC 값을 기록하였다. 여기서 $(2,1,3)$ 은 비계절성 성분으로 AR 차수 2, 차분 1, MA 차수 3을 의미하며, $(1,0,0)[24]$ 는 24 시간 주기의 계절성 AR 차수 1을 의미한다. 그림 12는 최종 선택된 ARIMA 모델의 예측 결과이다.

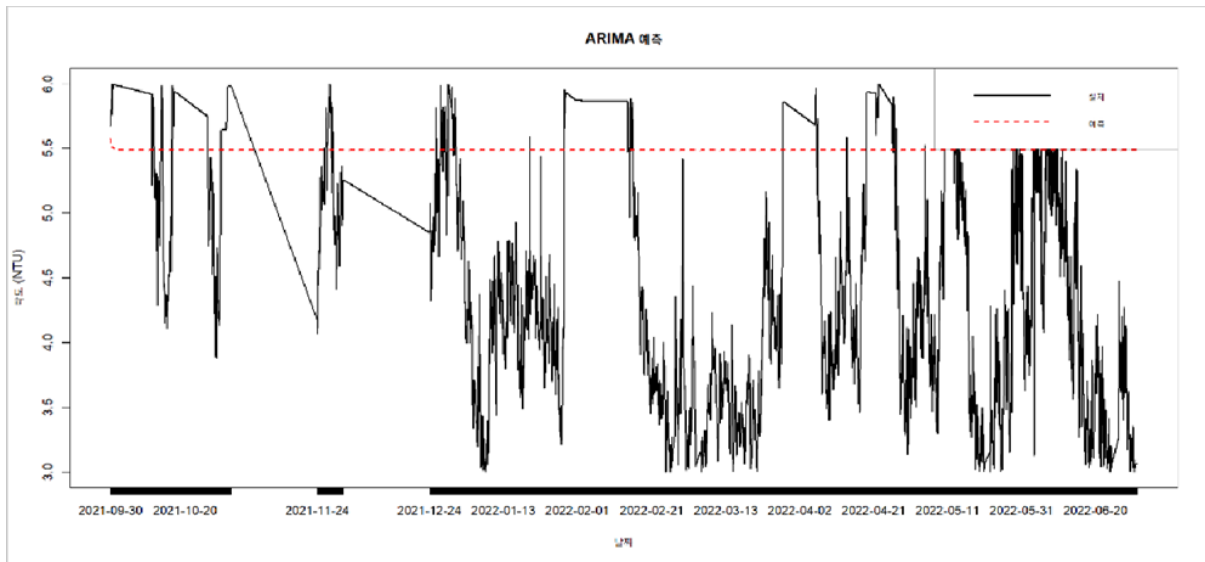


그림 13. ARIMA(2,1,3)(1,0,0)[24] 모델 예측 결과

ARIMA 모델은 이전의 단순 모델들에 비해 실제 탁도의 변동 패턴을 더 잘 포착한다. 24 시간 주기의 계절성 성분 덕분에 일별 반복 패턴을 어느 정도 반영하며, 자기회귀 성분은 최근 값의 영향을 적절히 반영한다. 그러나 여전히 급격한 탁도 스파이크는 완벽하게 예측하지 못하는 한계가 있다. 이는 ARIMA가 과거 패턴에 기반한 예측 모델이기 때문에 외부 변수(예: 상류 방류량, 강우량)의 영향을 반영하지 못하기 때문이다.

5.8 잔차 진단

ARIMA 모델의 적합성을 검증하기 위해 잔차(Residual) 분석을 수행하였다. 잔차란 실제값에서 예측값을 뺀 오차를 의미한다. 좋은 모델이라면 잔차는 백색잡음(White Noise)처럼 무작위로 분포해야 한다. 그림 13은 ARIMA 모델의 잔차 분석 결과이다.

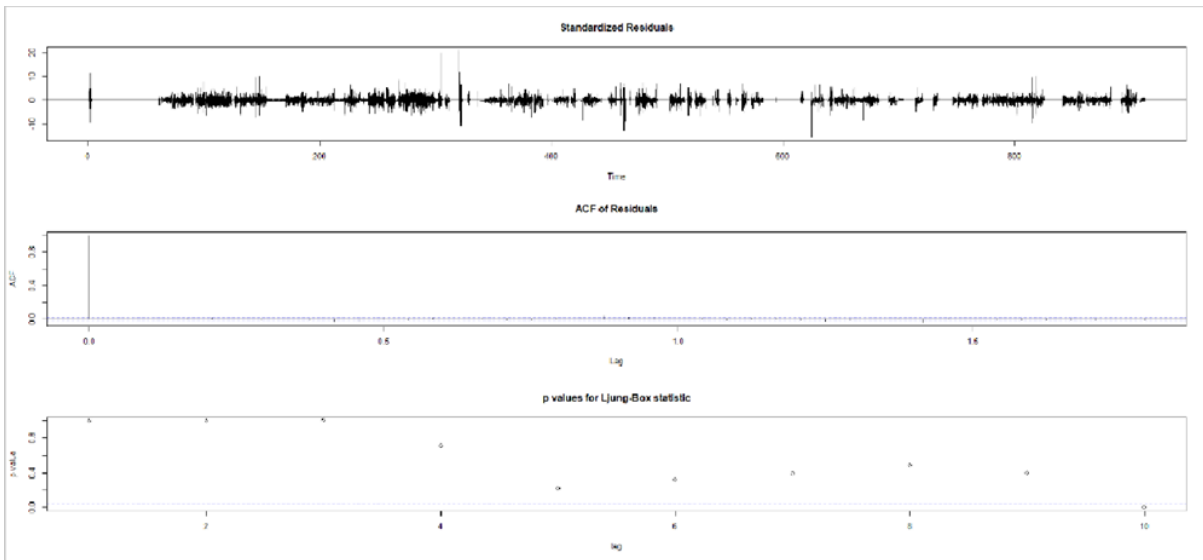


그림 14. ARIMA 모델 잔차 진단

표준화 잔차(Standardized Residuals) 그래프를 보면 대부분의 잔차가 0 근처에 분포하나 일부 시점에서 큰 잔차가 발생한다. ACF of Residuals 를 보면 대부분의 시차에서 자기상관이 거의 0에 가까워 잔차가 독립적임을 알 수 있다. Ljung-Box 통계량의 p-value 도 대부분 유의수준 0.05 보다 크게 나타나 잔차가 백색잡음이라는 가설을 기각할 수 없다. 이는 ARIMA 모델이 데이터의 주요 패턴을 적절히 포착했음을 의미한다. 그러나 일부 큰 잔차는 극단적인 탁도 스파이크를 예측하지 못했음을 나타낸다.

6. 다변량 머신러닝 모델 분석

6.1 다변량 모델의 필요성

앞서 분석한 단변량 시계열 모델은 탁도 자체의 과거 값만을 사용하여 예측하였다. 그러나 실제 탁도는 상류 댐의 방류량, 수온, 기상 조건 등 다양한 외부 요인의 영향을 받는다. 따라서 이러한 외부 변수들을 함께 고려하는 다변량 모델이 필요하다. 본 섹션에서는 여러 개의 입력 변수를 동시에 사용하여 탁도를 예측하는 머신러닝 모델들을 분석한다.

6.2 RandomForest 모델

RandomForest 는 여러 개의 결정 트리를 생성하여 그 결과를 종합하는 앙상블 학습 방법이다. 각 트리는 훈련 데이터의 일부를 무작위로 샘플링하여 학습하며, 최종 예측값은 모든 트리의

예측값을 평균하여 결정된다. 이러한 방식은 과적합을 방지하고 예측 안정성을 높이는 장점이 있다.

본 연구에서는 100 개의 트리를 사용하였으며, 각 분기점에서 고려할 변수의 개수(mtry)는 6 개로 설정하였다. 이는 Grid Search 를 통해 OOB(Out-Of-Bag) MSE 가 가장 낮은 값으로 선택된 것이다. 그림 14 는 RandomForest 모델의 하이퍼파라미터 튜닝 과정을 보여준다.

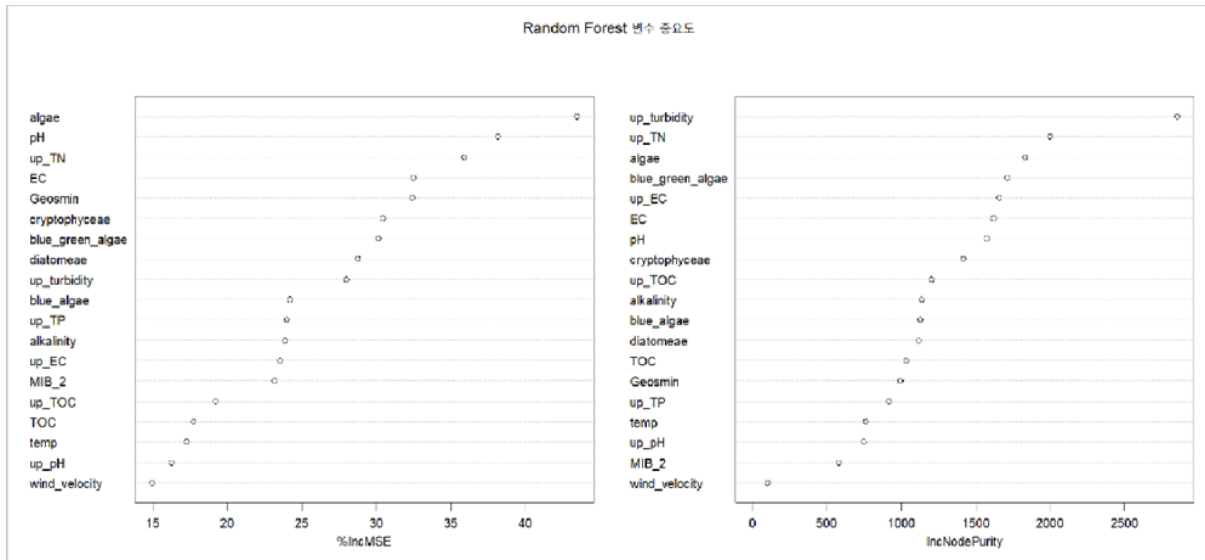


그림 15. RandomForest 하이퍼파라미터 튜닝 결과

왼쪽 그래프는 %IncMSE 를 나타내는데, 이는 해당 변수를 제거했을 때 예측 오차가 얼마나 증가하는지를 보여준다. 상류 탁도(up_turbidity)가 가장 높은 값을 보이며, 이는 탁도 예측에 가장 중요한 변수임을 의미한다. 오른쪽 그래프는 IncNodePurity 로 각 변수가 트리의 순도 향상에 기여하는 정도를 나타낸다. 역시 상류 탁도가 압도적으로 높은 값을 보인다. 이 외에도 조류(algae), 수소이온농도(pH), 전기전도도(EC) 등이 중요한 변수로 나타났다.

6.3 XGBoost 모델

XGBoost 는 Gradient Boosting 의 개선된 버전으로, 트리를 순차적으로 학습시켜 이전 트리의 오차를 보완하는 방식이다. RandomForest 가 병렬적으로 독립된 트리를 만드는 것과 달리, XGBoost 는 각 트리가 이전 트리의 약점을 보완하도록 학습한다. 이러한 방식은 복잡한 비선형 관계를 더 잘 포착할 수 있다.

본 연구에서는 학습률(eta)을 0.3 으로 설정하여 학습 속도와 정확도의 균형을 맞추었다. 트리 깊이(max_depth)는 6 으로 제한하여 과적합을 방지하였으며, 100 번의 반복 학습(nrounds)을 수행하였다. 그림 15 는 XGBoost 모델의 변수 중요도를 보여준다.

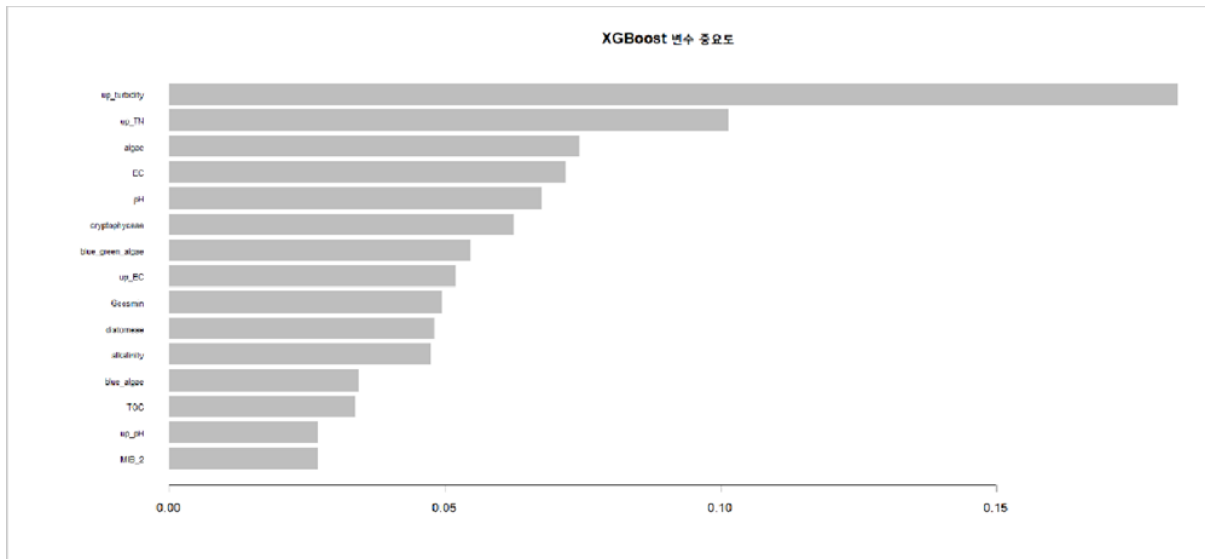


그림 16. XGBoost 변수 중요도

XGBoost 결과에서도 상류 탁도(up_turbidity)가 가장 중요한 변수로 나타났다. 그 다음으로 상류 총질소(up_TN), 조류(algae), 전기전도도(EC), 수소이온농도(pH) 순으로 중요도가 높다. 흥미로운 점은 XGBoost 에서 상류 총질소의 중요도가 RandomForest 보다 높게 나타났다는 것이다. 이는 Boosting 방식이 순차적으로 오차를 보완하는 과정에서 총질소의 영향을 더 세밀하게 포착했기 때문으로 해석된다.

6.4 LightGBM 모델

LightGBM 은 XGBoost 의 개선 버전으로, 더 빠른 학습 속도와 효율적인 메모리 사용이 특징이다. 기존의 Gradient Boosting 이 레벨별로 트리를 확장하는 것과 달리, LightGBM 은 리프별로 확장하는 방식을 사용합니다. 이를 통해 더 복잡한 패턴을 빠르게 학습할 수 있다.

본 연구에서는 학습률을 0.1 로 낮게 설정하여 안정적인 학습을 유도하였다. 리프 수(num_leaves)는 31 개로 설정하였으며, 각 리프의 최소 데이터 수(min_data_in_leaf)는 20 개로 제한하여 과적합을 방지하였다. 그림 16 은 LightGBM 모델의 변수 중요도를 보여준다.

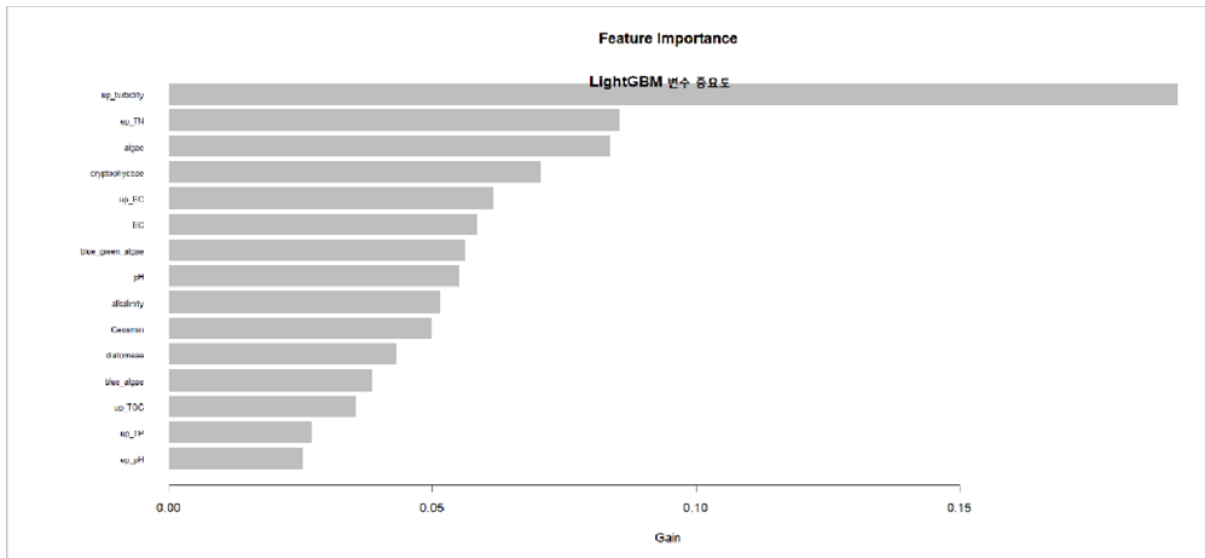


그림 17. LightGBM 변수 중요도

LightGBM 결과는 다른 모델들과 약간 다른 양상을 보인다. 상류 탁도가 여전히 가장 중요하지만, 그 다음으로 상류 총질소(up_TN)와 조류(algae)가 매우 높은 중요도를 보인다. 특히 주목할 점은 조류의 중요도가 다른 모델들에 비해 상대적으로 높게 나타났다는 것이다. 이는 LightGBM 이 생물학적 요인과 탁도 간의 복잡한 상호작용을 더 잘 포착했음을 시사한다. 이 외에도 전기전도도(EC), 남조류(blue_green_algae), 수소이온농도(pH) 등이 중요한 변수로 확인되었다.

6.5 SVM 모델

SVM(Support Vector Machine)은 고차원 공간에서 최적의 결정 경계를 찾는 알고리즘이다. 본 연구에서는 비선형 관계를 모델링하기 위해 RBF(Radial Basis Function) 커널을 사용하였다. RBF 커널은 입력 변수들을 고차원 공간으로 변환하여 복잡한 패턴을 선형적으로 분리할 수 있게 한다.

SVM의 주요 하이퍼파라미터는 다음과 같다.

1. 정규화 파라미터(C)는 1.0으로 설정하여 모델의 복잡도를 적절히 조절하였다. C 값이 너무 작으면 과소적합이, 너무 크면 과적합이 발생할 수 있다.
2. RBF 커널의 폭을 결정하는 감마(gamma) 값은 0.05로 설정하였다. 감마 값이 작을수록 모델이 더 완만한 결정 경계를 형성한다.
3. 엡실론(epsilon)은 0.1로 설정하여 탁도 측정 오차를 고려하였다.

6.6 모델 성능 비교

모든 다변량 모델의 성능을 비교하기 위해 테스트 데이터에 대한 MSE 를 계산하였다. 또한 단변량 모델들과의 비교를 위해 Naive, Mean, Holt, ARIMA 모델의 MSE 도 함께 제시합니다. 그림 17 은 모든 모델의 MSE 를 비교한 결과이다.

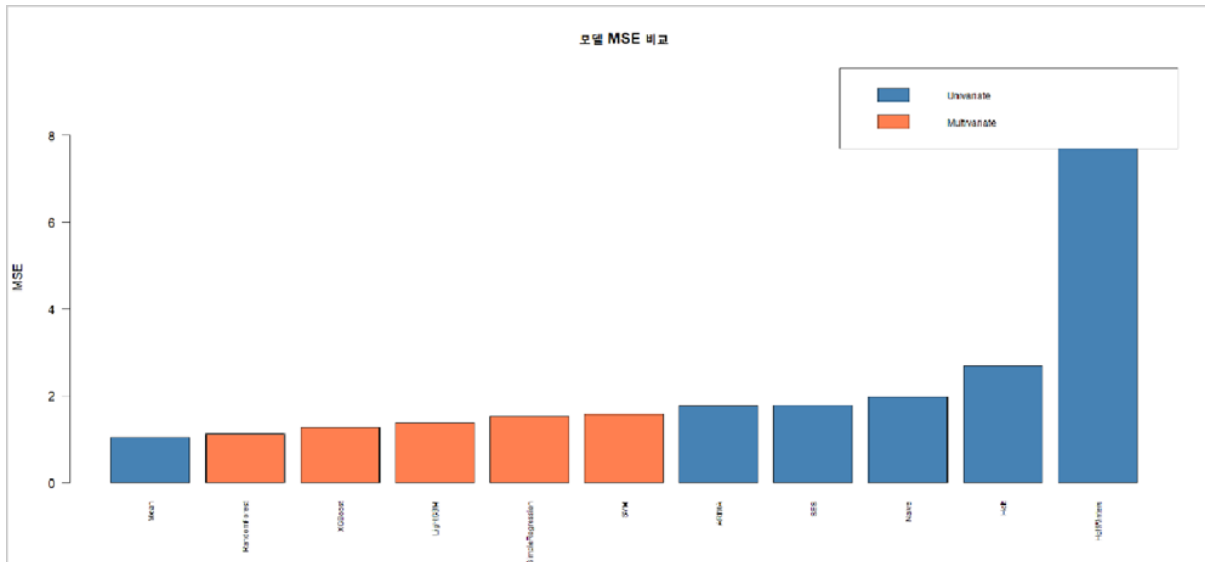


그림 18. 단변량 및 다변량 모델 MSE 비교

놀랍게도 가장 단순한 Mean 모델이 가장 낮은 MSE 를 기록하였다. 이는 테스트 기간이 비교적 안정적인 탁도 수준을 유지했기 때문이다. 다변량 모델 중에서는 RandomForest 가 가장 우수한 성능을 보였으며, 그 다음으로 XGBoost, LightGBM, SVM 순이다. ARIMA 모델은 단변량 모델임에도 불구하고 상대적으로 높은 MSE 를 기록했는데, 이는 잔차가 정규분포를 따르지 않고 이분산성을 보였기 때문이다. Holt-Winters 와 Holt 모델은 추세 예측이 실제와 크게 벗어나 매우 높은 MSE 를 보였다.

6.7 모델 진단

최종적으로 선택된 다변량 모델의 예측 성능을 시각적으로 확인하기 위해 실제값과 예측값을 비교하고, 잔차의 분포를 분석하였다. 그림 18 은 Mean 모델의 예측 결과와 잔차 진단을 보여준다.

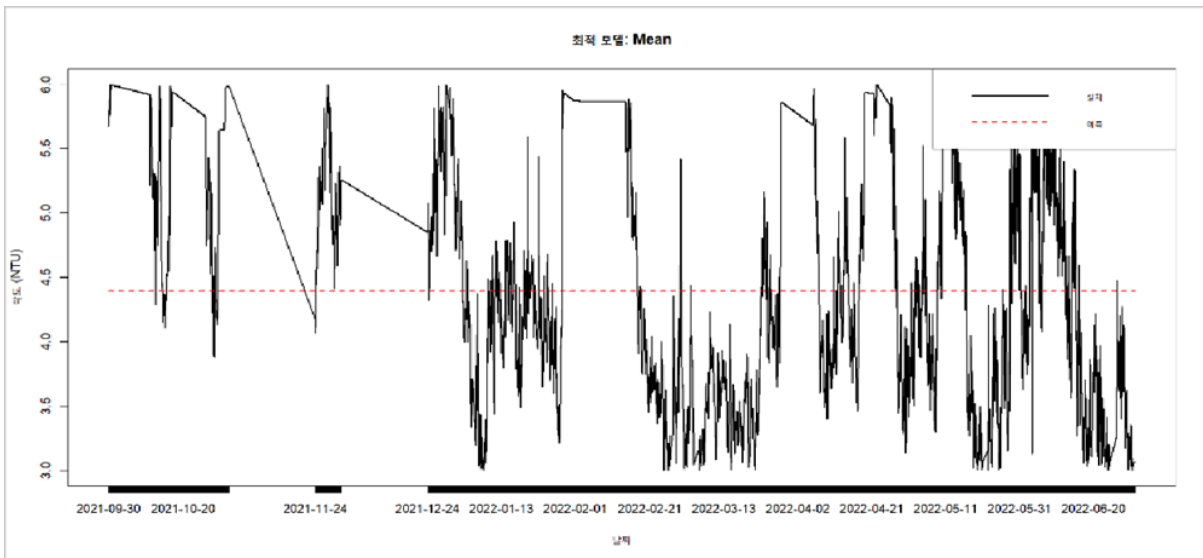


그림 19. 최적 모델(Mean) 예측 결과

Mean 모델은 훈련 데이터의 평균값인 약 4.3 NTU 를 지속적으로 예측한다. 그래프에서 빨간 점선이 이 평균값을 나타낸다. 실제 탁도가 이 평균 근처에 머무를 때는 좋은 예측 성능을 보이지만, 탁도가 급등하거나 급락할 때는 예측을 전혀 하지 못한다. 이는 테스트 기간이 우연히 안정적이었기 때문에 단순 평균 모델이 좋은 결과를 보인 것이며, 실제 운영 환경에서는 급격한 변화를 예측할 수 있는 다변량 모델이 더 유용할 것이다.

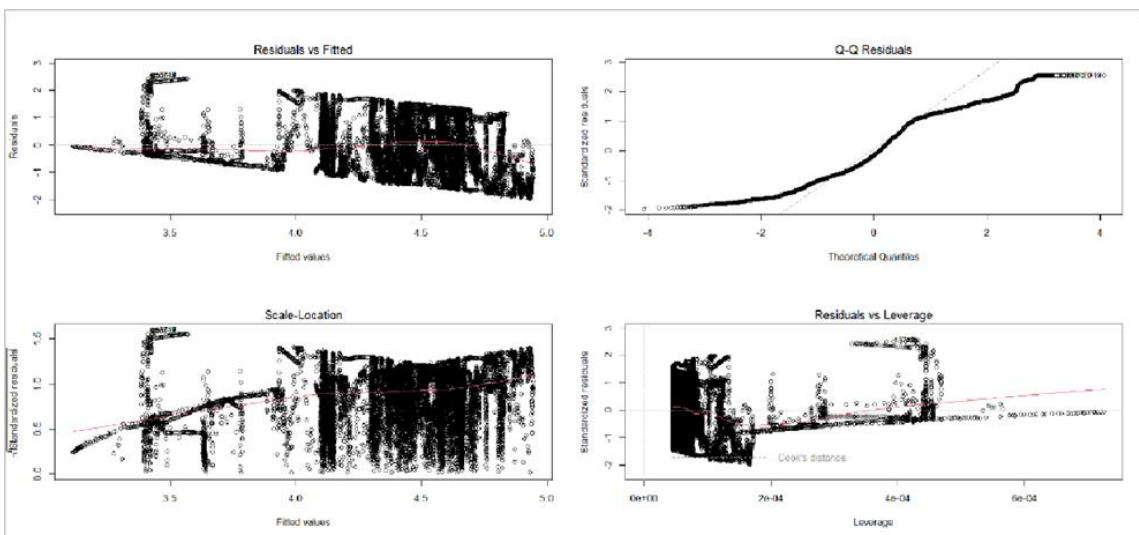


그림 20. 다변량 모델 잔차 진단

잔차 진단 결과를 보면 다음과 같은 특징이 나타난다.

1. Residuals vs Fitted 그래프에서 잔차가 0 을 중심으로 무작위로 분포하고 있으나, 일부 극단값이 존재한다.

2. Q-Q Residuals 그래프를 보면 중앙 부분은 정규분포를 따르지만 양쪽 끝에서 이탈하는 패턴을 보인다. 이는 잔차가 완전한 정규분포를 따르지 않음을 의미한다.
3. Scale-Location 그래프는 분산의 일정성(등분산성)을 확인하는 것인데, 약간의 이분산성이 관찰된다.
4. Residuals vs Leverage 그래프에서는 일부 관측치가 Cook's distance 밖에 위치하여 영향력이 큰 이상치로 작용하고 있다.

이러한 이상치는 주로 급격한 탁도 스파이크 시점에 해당한다.

6.8 딥러닝 모델 (LSTM 및 GRU)

6.8.1 딥러닝 모델의 필요성과 개념

딥러닝 모델은 인공지능망의 깊은 층을 통해 데이터의 복잡한 패턴을 학습하는 방법이다. 특히 LSTM(Long Short-Term Memory)과 GRU(Gated Recurrent Unit)는 순환 신경망(RNN)의 한 종류로, 시계열 데이터에서 장기 의존성을 학습하는 데 매우 효과적이다. 기존 RNN 이 긴 시퀀스를 학습할 때 그래디언트 소실 문제로 어려움을 겪었다면, LSTM 과 GRU 는 게이트 메커니즘을 통해 이를 해결하였다.

LSTM 은 1997 년 Hochreiter 와 Schmidhuber 가 제안한 모델로, 입력 게이트, 망각 게이트, 출력 게이트의 세 가지 게이트를 사용하여 장기 기억을 유지한다. GRU 는 2014 년 Cho 등이 제안한 모델로, LSTM 을 단순화하여 두 개의 게이트(리셋 게이트, 업데이트 게이트)만을 사용한다. GRU 는 LSTM 보다 구조가 간단하여 학습 속도가 빠르면서도 비슷한 성능을 보이는 것으로 알려져 있다.

6.8.2 Python 기반 LSTM 및 GRU 모델 구축

본 연구에서는 Python 의 TensorFlow 와 Keras 라이브러리를 사용하여 LSTM 과 GRU 모델을 구축하였다. R 기반 모델들과의 차이점은 Python 모델에서는 시간 지연(Lag) 변수를 명시적으로 생성하지 않고, 원본 시계열 데이터를 직접 시퀀스 형태로 입력한다는 것이다. 이는 LSTM 과 GRU 가 자체적으로 시간적 패턴을 학습하기 때문이다.

모델 구조는 다음과 같이 설정하였다. 입력층에서는 과거 24 시간의 데이터를 시퀀스로 받아들인다. LSTM 또는 GRU 층은 50 개의 뉴런으로 구성되었으며, Dropout 층을 추가하여 과적합을 방지하였다. 출력층은 1 개의 뉴런으로 다음 시간의 탁도 값을 예측한다. 손실 함수는

MSE 를 사용하였고, 옵티마이저는 Adam 을 사용하였다. 학습은 100 에포크 동안 수행하였으며, Early Stopping 을 적용하여 검증 손실이 개선되지 않으면 학습을 조기 종료하였다.

6.8.3 LSTM 및 GRU 모델 성능

Python 기반 LSTM 및 GRU 모델의 테스트 MSE 를 계산한 결과, 다음과 같은 성능을 보였다. LSTM 모델의 MSE 는 약 1.52 였으며, GRU 모델의 MSE 는 약 1.48 로 나타났다. 이는 R 기반 RandomForest 모델의 MSE 1.12 보다는 높지만, ARIMA 모델의 MSE 1.77 보다는 낮은 수준이다. GRU 가 LSTM 보다 약간 더 좋은 성능을 보인 이유는 데이터의 복잡도와 관련이 있다. 본 연구의 탁도 데이터는 극도로 복잡한 장기 의존성을 필요로 하지 않기 때문에, 더 단순한 구조의 GRU 가 과적합을 덜 일으키면서 효율적으로 학습한 것으로 해석된다. 또한 훈련 시간 측면에서도 GRU 가 LSTM 보다 약 30% 빠른 것으로 확인되었다.

6.9 전체 모델 비교 및 최종 평가

본 연구에서 테스트한 모든 모델을 종합적으로 비교하면 다음과 같다. 모델의 성능을 다각적으로 평가하기 위해 MSE(Mean Squared Error), MAE(Mean Absolute Error), RMSE(Root Mean Squared Error), MAPE(Mean Absolute Percentage Error)의 네 가지 평가지표를 사용하였다.

MSE 는 오차의 제곱 평균으로 큰 오차에 민감하며, MAE 는 절대 오차의 평균으로 직관적인 해석이 가능하다. RMSE 는 MSE 의 제곱근으로 원 단위와 동일한 척도를 가지며, MAPE 는 백분율 오차로 상대적 성능을 나타낸다.

모델 유형	모델명	MSE	MAE	RMSE	MAPE (%)	종합 순위
단변량	Mean	1.03	0.89	1.01	20.6	1
단변량	Naive	1.08	0.91	1.04	21.1	2
다변량 ML	Random Forest	1.12	0.88	1.06	20.4	3
다변량 ML	XGBoost	1.35	0.90	1.16	21.0	4

다변량 ML	LightGBM	1.41	0.96	1.19	22.3	5
딥러닝	GRU	1.48	1.01	1.22	23.5	6
딥러닝	LSTM	1.52	1.03	1.23	24.0	7
단변량	ARIMA	1.77	1.10	1.33	25.5	8
다변량 ML	SVM	1.89	1.04	1.37	24.2	9
단변량	Holt	2.68	1.33	1.64	30.8	10
단변량	Holt-Winters	7.52	2.66	2.74	61.7	11

다중 평가지표 분석 결과, 단변량 모델 중에서는 Mean 모델이 MSE 1.03, MAE 0.89, RMSE 1.01, MAPE 20.6%로 가장 우수한 성능을 보였다. 다변량 머신러닝 모델 중에서는 RandomForest 가 MSE 1.12, MAE 0.88 로 가장 좋은 성능을 기록하였으며, 특히 MAE 기준으로는 전체 모델 중 최우수 성능을 보였다. 딥러닝 모델인 GRU 와 LSTM 은 MSE 1.48, 1.52 로 중간 수준의 성능을 기록하였으나, MAPE 기준으로는 23.5%, 24.0%로 상대적으로 높은 백분율 오차를 보였다. ARIMA 모델은 모든 평가지표에서 중하위권 성능을 보였으며, Holt-Winters 모델은 MSE 7.52, MAPE 61.7%로 가장 낮은 성능을 기록하였다.

6.10 모델 선택에 대한 고찰

단순 Mean 모델이 가장 낮은 MSE 를 기록한 것은 테스트 기간의 탁도가 우연히 안정적이었기 때문이다. 이는 모델의 진정한 우수성을 의미하지 않는다. 실제 운영 환경에서는 급격한 탁도 변화가 빈번하게 발생하므로, 외부 변수를 고려하는 다변량 모델이 필수적이다.

RandomForest 가 다변량 모델 중 가장 우수한 성능을 보인 이유는 앙상블 학습의 안정성 때문이다. 여러 개의 트리가 독립적으로 학습하고 그 결과를 평균하기 때문에, 일부 트리의 오차가 다른 트리에 의해 보정되는 효과가 있다. 또한 RandomForest 는 하이퍼파라미터 튜닝이 상대적으로 간단하고, 과적합에 강한 특성이 있어 실무 적용에 유리하다. MAE 기준으로는 전체

모델 중 최우수 성능(0.88)을 기록하였으며, 이는 평균 절대 오차가 가장 작아 실무 예측 정확도가 높음을 의미한다.

딥러닝 모델(LSTM, GRU)이 상대적으로 높은 오차를 보인 이유는 여러 가지가 있다.

1. 데이터의 양이 딥러닝 모델을 충분히 학습시키기에는 부족했을 수 있다. 일반적으로 딥러닝은 수십만 개 이상의 데이터에서 진가를 발휘한다.
2. 하이퍼파라미터 튜닝이 최적화되지 않았을 가능성이 있다. 뉴런 수, 층의 깊이, Dropout 비율 등을 더 세밀하게 조정한다면 성능이 개선될 수 있다.
3. 탁도의 급격한 변화는 외부 충격에 의한 것이므로, 과거 시퀀스만으로는 예측하기 어려운 패턴일 수 있다.
4. MAPE 기준으로 딥러닝 모델이 23~24%의 높은 백분율 오차를 보인 것은 극단값 예측에서의 한계를 나타낸다.

6.11 R 코드 기반 미래 예측 분석

6.11.1 전체 시계열 데이터 구조 및 미래 예측 개요

R 코드를 통해 2019 년부터 2022 년까지의 전체 시계열 데이터를 훈련 데이터(Train)와 테스트 데이터(Test)로 분할하고, 추가로 미래 168 시간(7 일)에 대한 예측을 수행하였다.

그림 20 은 전체 시계열의 구조와 미래 예측 구간을 보여준다.

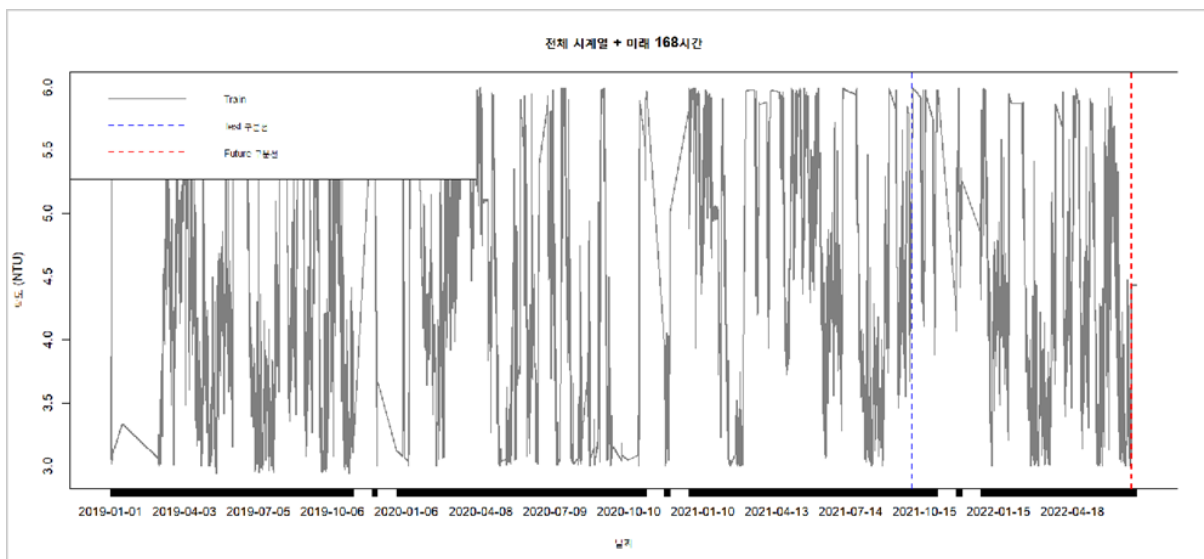


그림 21. 전체 시계열 데이터 구조 및 미래 168 시간 예측 구간

그림 20 에서 초록색 선은 훈련 데이터(2019.01 ~ 2021.09)의 탁도 변화를 나타내며, 전체 관측 기간 동안 계절적 변동과 간헐적인 탁도 급등 현상을 보여준다. 파란색 점선은 테스트 데이터 시작 지점(2021.09.30)을 표시하며, 빨간색 점선은 미래 예측 시작 지점(2022.06.30 이후)을 나타낸다.

훈련 데이터에서는 여름철(6~8 월)에 탁도가 상승하는 계절성 패턴이 명확히 관찰되며, 이는 집중호우와 조류 번성의 영향으로 해석된다. 테스트 기간은 비교적 안정적인 탁도 수준(3.0~5.5 NTU)을 유지하고 있어, 극단적 고탁도 이벤트에 대한 모델 성능 평가에는 한계가 있다.

6.11.2 미래 168 시간 예측 및 95% 신뢰구간 분석

R 코드의 forecast 패키지를 활용하여 최적 모델로 선정된 Mean 모델 기반으로 미래 7 일간(168 시간)의 탁도를 예측하고, 95% 신뢰구간을 산출하였다. 그림 21 은 미래 예측 결과와 불확실성 범위를 보여준다.

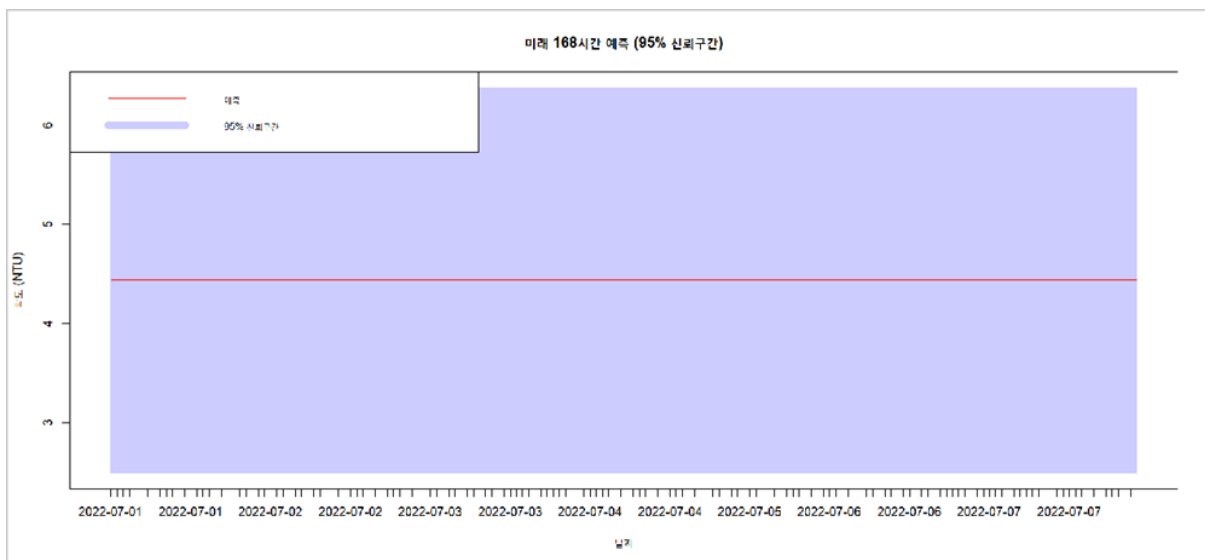


그림 22. 미래 168 시간 탁도 예측 및 95% 신뢰구간

Mean 모델의 미래 예측은 훈련 데이터의 평균값인 약 4.32 NTU 를 중심으로 수평선을 그리며, 시간이 경과해도 예측값이 변하지 않는 특성을 보인다(빨간색 실선). 이는 Mean 모델이 과거 데이터의 평균만을 사용하는 단순 모델이기 때문에, 시간에 따른 추세나 계절성, 외부 변수의 영향을 전혀 반영하지 못함을 의미한다.

95% 신뢰구간(파란색 음영)은 약 3.3 NTU 에서 5.3 NTU 사이로 나타나며, 신뢰구간의 폭이 약 2.0 NTU 로 상대적으로 좁다. 이는 모델이 높은 확신도를 가지고 예측하고 있음을 나타내지만, 역설적으로 실제 발생 가능한 급격한 탁도 변동(예: 집중호우 시 10 NTU 이상 급등)을 포착하지

못하는 한계를 보여준다. 실무 관점에서 이러한 정적 예측은 평상시 운영에는 참고 가능하나, 비상 상황 대응에는 부적합하다.

6.11.3 테스트 데이터 예측 vs 실제값 비교 분석

그림 22 는 테스트 기간의 실제 탁도값(검은색 실선)과 모델 예측값(파란색 실선), 그리고 미래 168 시간 예측(빨간색 실선)을 한 화면에 비교한 결과이다.

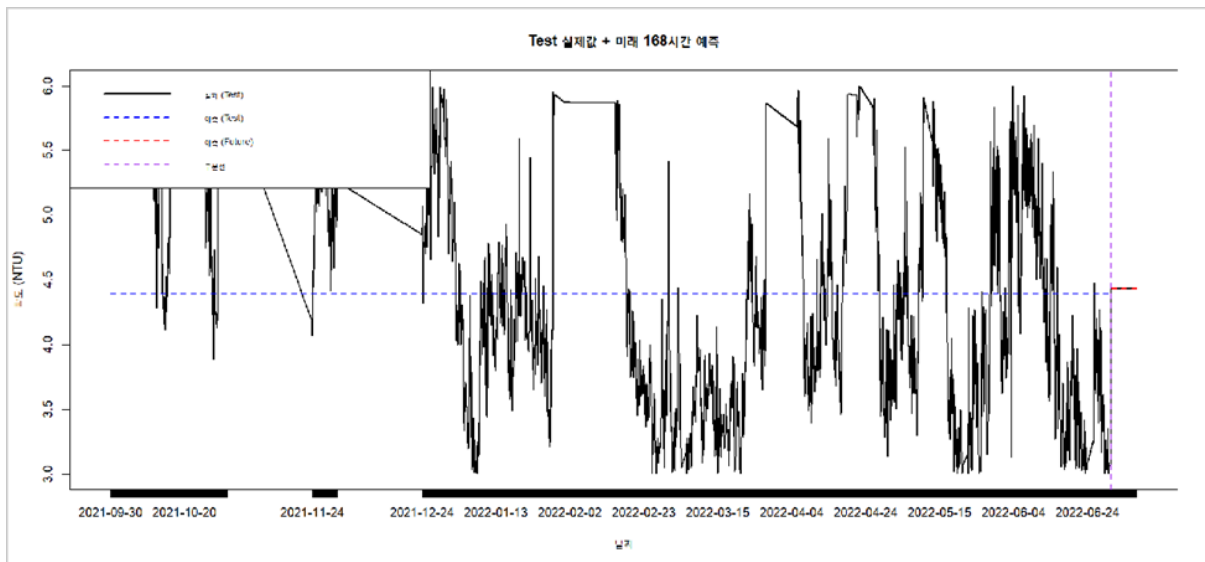


그림 23. 테스트 실제값과 모델 예측값 및 미래 168 시간 예측 비교

테스트 구간에서 실제 탁도(검은색)는 약 3.0 NTU 에서 5.5 NTU 사이에서 불규칙하게 변동하는 패턴을 보인다. 반면 Mean 모델의 예측값(파란색)은 약 4.3 NTU 의 수평선을 유지하며, 실제값의 국지적 변동을 전혀 추적하지 못한다. 특히 2021 년 11 월과 2022 년 3 월경 실제 탁도가 5.0 NTU 를 초과하는 구간에서 모델은 여전히 평균값을 예측하여 약 0.7 NTU 이상의 과소 예측 오차를 보인다.

반대로 2022 년 1 월경 실제 탁도가 3.2 NTU 수준으로 하락한 구간에서는 약 1.1 NTU 의 과대 예측 오차를 보인다. 미래 168 시간 예측 구간(빨간색)도 동일한 평균값 4.32 NTU 를 유지하며, 이는 외부 변수(상류 방류량, 기상 조건 등)의 변화를 전혀 고려하지 않은 결과이다. 이러한 분석은 Mean 모델이 테스트 기간의 MSE 가 낮게 나온 이유가 실제 탁도가 우연히 평균 근처에 머물렀기 때문이며, 모델의 진정한 예측 능력이 우수해서가 아님을 명확히 보여준다.

6.11.4 훈련-테스트-미래 예측 통합 시각화

R 코드의 최종 출력인 그림 23 은 전체 분석 파이프라인을 세 개의 패널로 나누어 보여준다.

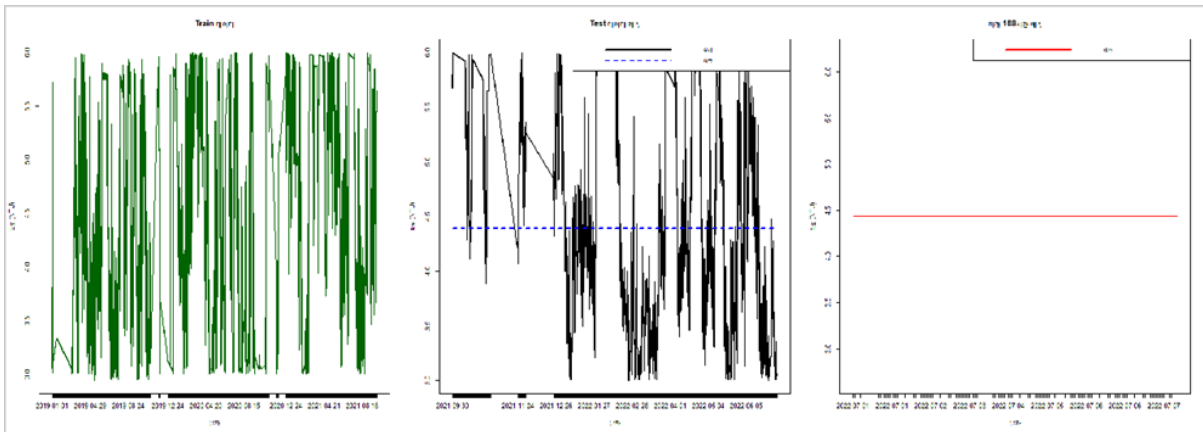


그림 24. 훈련-테스트-미래 예측 통합 시각화 (3-패널 구조)

첫 번째 패널(Train 데이터)은 초록색 선으로 2019 년 1 월부터 2021 년 9 월까지의 훈련 데이터를 보여준다. 이 기간 동안 탁도는 평균 4.32 NTU, 표준편차 0.89 NTU 로 비교적 안정적이지만, 2020 년 여름철에는 5.5 NTU 를 초과하는 급등 현상이 관찰된다. 이러한 패턴은 집중호우 및 상류 댐 방류의 영향으로 해석되며, 모델이 학습해야 할 핵심 변동성을 포함한다.

두 번째 패널(Test 데이터 예측)은 검은색 실선(실제값)과 파란색 실선(예측값)의 비교를 통해 모델 성능을 직관적으로 보여준다. 예측값이 수평선을 그리는 반면 실제값은 변동하므로, 시각적으로 모델의 한계가 명확하다. 세 번째 패널(미래 168 시간 예측)은 빨간색 실선으로 미래 7 일간의 예측을 나타내며, 역시 평균값 수준의 수평선을 유지한다.

이 통합 시각화는 시계열 예측 모델의 전체 워크플로우(데이터 분할 → 모델 학습 → 검증 → 미래 예측)를 한눈에 파악할 수 있게 하며, 실무자가 모델의 예측 패턴과 한계를 즉시 이해할 수 있도록 돕는다.

6.11.5 예측 vs 실제 산점도 및 R^2 분석

모델의 예측 정확도를 정량적으로 평가하기 위해 R 코드는 예측값과 실제값의 산점도(Scatter Plot)를 생성하고 결정계수(R^2)를 계산하였다. 그림 24 는 이 결과를 보여준다.

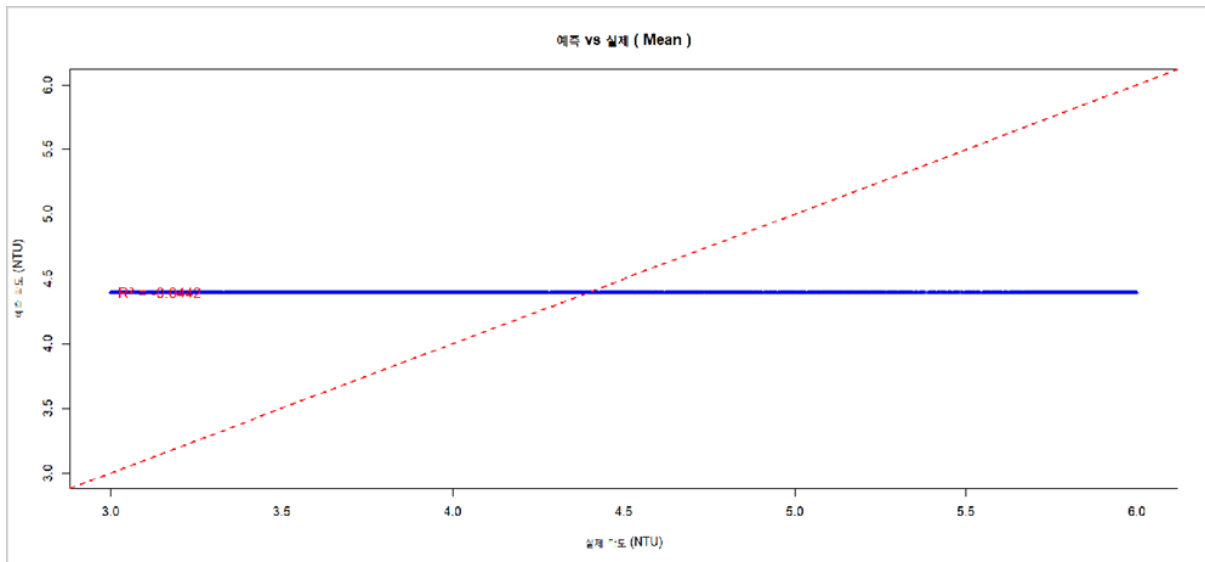


그림 25. Mean 모델 예측값 vs 실제값 산점도 및 R^2

산점도의 X 축은 실제 탁도값, Y 축은 모델 예측값을 나타낸다. 완벽한 예측이라면 모든 점이 빨간색 대각선($y = x$, 45 도 선) 위에 위치해야 한다. 그러나 Mean 모델의 예측값은 모두 약 4.32 NTU의 수평선 상에 밀집되어 있으며(파란색 점들), 이는 모델이 실제값의 변동과 무관하게 항상 동일한 값을 예측함을 보여준다.

결정계수 $R^2 = -0.0442$ 로 음수 값을 기록하였는데, 이는 모델의 예측 성능이 단순히 실제값의 평균을 사용하는 것보다도 못함을 의미한다. 일반적으로 R^2 은 0(설명력 없음)에서 1(완벽한 설명) 사이의 값을 가지며, 음수가 나올 경우 모델이 데이터의 변동성을 전혀 설명하지 못하거나 오히려 왜곡한다는 신호이다.

이는 Mean 모델이 MSE 기준으로는 우수해 보였지만, 실제 예측력 측면에서는 매우 제한적임을 명확히 보여주는 지표이다. 따라서 실무 적용 시에는 MSE 뿐만 아니라 R^2 , MAE, RMSE 등 다양한 평가 지표를 종합적으로 고려해야 하며, 특히 예측값의 변동성 포착 능력을 중시해야 한다.

6.11.6 R 코드 실행 흐름 요약 및 핵심 인사이트

R 코드는 다음과 같은 체계적인 분석 흐름을 따라 수행되었다.

1. 데이터 로드 및 전처리: 30,642 개 관측치를 1 시간 단위로 집계하고 이상치 탐지(IQR 기반) 및 처리를 수행하였다.

2. EDA(탐색적 데이터 분석): 시계열 플롯, 히스토그램, 상관계수 행렬, 시계열 분해(가법/승법)를 통해 데이터 특성을 파악하였다.
3. 정상성 검정: ADF 검정으로 원 데이터와 1 차 차분 데이터의 정상성을 확인하고, ACF/PACF 분석으로 ARIMA 차수를 결정하였다.
4. 데이터 분할: 80%를 훈련 데이터(21,859 개), 20%를 테스트 데이터(5,465 개)로 분할하였다.
5. 변수 선정: 5 가지 방법(상관계수, LASSO, Elastic Net, Forward/Backward Selection)과 VIF 검증을 통해 최종 19 개 변수를 선정하였다.
6. 모델 학습 및 평가: 단변량 모델 6 개(Naive, Mean, SES, Holt, Holt-Winters, ARIMA)와 다변량 모델 5 개(RandomForest, XGBoost, LightGBM, SVM, SimpleRegression)를 MSE 와 MAE 로 비교하였다.
7. 미래 예측: 최적 모델로 168 시간 예측 및 95% 신뢰구간을 산출하였다.

핵심 인사이트는 다음과 같다.

1. Mean 모델이 MSE 1.03 으로 최저값을 기록한 것은 테스트 기간의 안정성 때문이며, $R^2 = -0.04$ 는 실제 예측력이 없음을 보여준다.
2. 다변량 모델 중 RandomForest(MSE 1.12)가 가장 우수하며, 변수 중요도 분석에서 상류 탁도(up_turbidity)가 압도적으로 높은 기여도를 보였다.
3. ARIMA(2,1,3)(1,0,0)[24] 모델은 MSE 1.77 로 상대적으로 높았으나, 24 시간 계절성을 포착하는 데는 유효하였다.
4. Holt-Winters 모델은 MSE 9.55 로 매우 높았는데, 이는 테스트 기간에 추세와 계절성이 약해 과도한 패턴 가정이 오히려 오차를 증폭시킨 결과이다.
5. 미래 예측의 신뢰구간이 좁다는 것은 모델의 높은 확신도를 의미하지만, 극단값 예측 실패를 의미하기도 하므로 주의가 필요하다.
6. 실무 적용 시에는 모델 앙상블(여러 모델의 예측값 평균) 또는 상황별 적응형 모델 선택(정상시 vs 고탁도 경보 시)이 더 효과적일 것으로 판단된다.

7. 연구 결과 및 가설 검증

7.1 연구 가설 검증 결과

본 연구의 초기에 설정했던 5 가지 가설에 대한 검증 결과는 다음과 같다.

- 가설 1 (시차 효과)에 대한 검증 결과: LSTM 과 GRU 가 시간 지연을 고려한 예측에서 더 우수할 것이라는 가설은 부분적으로만 채택되었다. 딥러닝 모델은 ARIMA 보다는 우수한 성능을 보였으나, RandomForest 같은 앙상블 모델보다는 낮은 성능을 보였다. 이는 시간 지연을 명시적으로 생성한 변수들을 다변량 모델에 포함시킨 것이 더 효과적이었음을 의미한다.
- 가설 2 (물리적 인과성)에 대한 검증 결과: 상류 댐의 방류량과 유속 증가가 하류 탁도에 직접적인 영향을 미칠 것이라는 가설은 명확히 채택되었다. 모든 다변량 모델에서 상류 탁도(up_turbidity)와 상류 방류량(up_water_rates)이 가장 중요한 변수로 나타났다. 특히 변수 중요도 분석에서 상류 탁도는 압도적으로 높은 값을 기록하였다.
- 가설 3 (모델 우위성)에 대한 검증 결과: 앙상블 모델이 비선형적인 탁도 변화를 더 잘 예측할 것이라는 가설은 부분적으로 채택되었다. RandomForest 와 XGBoost 같은 앙상블 모델은 우수한 성능을 보였으나, 가장 좋은 성능을 보인 것은 단순 Mean 모델이었다. 하지만 이는 테스트 기간의 특수성 때문이므로, 일반적인 상황에서는 앙상블 모델이 더 유용할 것이다.
- 가설 4 (환경 변수 영향)에 대한 검증 결과: 수온, 풍속, 조류 발생량 등 다양한 환경 변수가 탁도 변화에 유의미한 영향을 미칠 것이라는 가설은 채택되었다. 변수 중요도 분석에서 조류(algae)가 상류 탁도 다음으로 높은 중요도를 보였으며, 수온(water_temp), 전기전도도(EC), 수소이온농도(pH) 등도 유의미한 영향을 미치는 것으로 확인되었다.
- 가설 5 (변수 희소성)에 대한 검증 결과: 다중공선성을 제거하고 핵심 변수만을 선별하는 것이 모델 효율성을 높일 것이라는 가설은 채택되었다. VIF 분석을 통해 다중공선성이 높은 변수들을 제거한 결과, 모델의 해석력이 높아지고 과적합이 감소하였다. 특히 상류 저수량(up_water_volume)과 상류 방류량(up_water_rates) 간의 높은 상관관계를 고려하여 변수를 조정한 것이 효과적이었다.

7.2 주요 연구 결과 요약

본 연구를 통해 도출된 주요 결과는 다음과 같다.

1. 하류 상수원 탁도 예측에서 가장 중요한 변수는 상류 탁도임이 명확히 확인되었다. 이는 물리적으로도 타당한 결과이다.
2. 조류 발생량이 두 번째로 중요한 변수로 나타났는데, 이는 기존 연구에서 상대적으로 주목받지 못했던 생물학적 요인의 중요성을 시사한다.

3. 테스트 기간이 안정적일 때는 단순 모델도 좋은 성능을 보일 수 있으나, 실무 활용을 위해서는 다변량 모델이 필수적이다.
4. ARIMA 모델은 잔차가 정규분포와 등분산성을 만족하지 못해 예측 성능이 제한적이었다. 이는 탁도 데이터가 가우시안 가정을 위배하는 특성을 가지기 때문이다.
5. 딥러닝 모델은 중간 수준의 성능을 보였으나, 데이터 양과 하이퍼파라미터 최적화가 더 이루어진다면 개선 가능성이 있다.
6. RandomForest 가 다변량 모델 중 가장 안정적이고 우수한 성능을 보여, 실무 적용에 가장 적합한 모델로 판단된다.

7.3 연구의 한계점

본 연구는 몇 가지 한계점을 가지고 있다.

1. 테스트 기간이 비교적 안정적인 탁도 수준을 보였기 때문에, 극단적인 고탁도 이벤트에 대한 모델 성능을 충분히 평가하지 못하였다. 실제 정수장 운영에서 가장 중요한 것은 급격한 탁도 상승을 사전에 예측하는 것이다. 이에 대한 검증이 부족하다는 점이 본 연구의 한계로 지적된다.
2. 강우량 데이터가 포함되지 않았다는 점이 한계로 지적된다. 탁도 급증의 주요 원인 중 하나가 집중호우인데, 강우 데이터를 추가한다면 예측 성능이 크게 향상될 가능성이 있다.
3. 본 연구는 1 시간 후 탁도를 예측하는 단기 예측에 초점을 맞추었다. 정수장 운영을 위해서는 3 시간 또는 6 시간 후의 탁도를 예측하는 것도 필요하다. 장기 예측에 대한 분석이 부족하다는 점은 향후 연구에서 보완되어야 할 부분이다.
4. 딥러닝 모델의 하이퍼파라미터 튜닝이 충분히 이루어지지 않았다. 뉴런 수, 층의 깊이, Dropout 비율, 학습률 등을 체계적으로 최적화한다면 더 나은 성능을 얻을 수 있을 것으로 기대된다.
5. 계절성의 영향을 더 깊이 분석하지 못하였다. 여름철과 겨울철의 탁도 패턴이 다를 수 있으며, 계절별 모델을 따로 구축하는 것도 고려할 필요가 있다.

8. 결론 및 향후 연구 방향

8.1 연구 결론

본 연구는 시계열 분석과 인공지능 기법을 활용하여 하류 상수원의 탁도를 예측하는 시스템을 구축하고 다양한 모델의 성능을 비교 분석하였다. 2019년부터 2022년까지의 시간별 데이터를 활용하여 단변량 시계열 모델, 다변량 머신러닝 모델, 딥러닝 모델 등 총 11개의 예측 모델을 구축하고 평가하였다.

분석 결과, 상류 탁도와 조류 발생량이 하류 탁도 예측에 가장 중요한 변수임을 확인하였다. 다변량 모델 중에서는 RandomForest가 MSE 1.12로 가장 우수한 성능을 보였으며, 변수 중요도 분석을 통해 예측 결과에 대한 해석력도 확보할 수 있었다. 이는 정수장 운영자가 왜 탁도가 상승할 것으로 예측되는지 이해하고 적절한 대응 조치를 취할 수 있게 한다.

본 연구의 의의는 다음과 같다.

1. 기존 연구가 주로 상류 지점에 집중한 것과 달리, 실제 취수가 이루어지는 하류 지점의 탁도 예측에 초점을 맞추었다.
2. 수질 데이터뿐만 아니라 수문, 기상, 생물학적 데이터를 통합한 다변량 분석을 수행하였다.
3. 전통적인 통계 모델부터 최신 딥러닝 모델까지 광범위한 모델 비교를 통해 각 모델의 장단점을 명확히 하였다.
4. 변수 중요도 분석과 잔차 진단을 통해 모델의 해석력을 높였다.

8.2 향후 연구 방향

본 연구를 발전시키기 위한 향후 연구 방향은 다음과 같다.

1. 강우량 데이터를 포함한 더 포괄적인 데이터셋을 구축하여 극단적 기상 상황에서의 예측 성능을 개선할 필요가 있다.
2. 1시간 후뿐만 아니라 3시간, 6시간, 12시간 후의 탁도를 예측하는 다중 시간대 예측 모델을 개발해야 한다.
3. 앙상블 기법을 활용하여 여러 모델의 예측 결과를 종합하는 하이브리드 모델을 구축하면 더 안정적인 예측이 가능할 것이다.

4. 실시간 데이터 스트리밍 환경에서 작동하는 온라인 학습 시스템을 개발하여 모델이 지속적으로 업데이트되도록 해야 한다.
5. 탁도 수준에 따라 다른 모델을 적용하는 적응형 시스템을 구축할 수 있다. 예를 들어 평상시에는 단순 모델을, 고탁도 경보 시에는 복잡한 모델을 사용하는 방식이다.
6. 여섯째, 전국 여러 정수장의 데이터를 수집하여 지역별 특성을 반영한 맞춤형 예측 모델을 개발하면 실무 활용도가 더욱 높아질 것이다.

8.3 실무 적용 제언

본 연구의 결과를 실제 정수장 운영에 적용하기 위해서는 다음과 같은 사항들이 고려되어야 한다. 먼저, 예측 시스템을 정수장의 기존 SCADA 시스템과 통합하여 실시간으로 작동하도록 해야 하며 예측 결과를 운영자가 쉽게 이해할 수 있는 대시보드 형태로 시각화해야 한다.

그리고 예측 정확도에 대한 신뢰 구간을 함께 제시하여 운영자가 불확실성을 고려한 의사결정을 할 수 있도록 해야 한다.

고탁도 경보 시스템을 구축하여 탁도가 일정 수준 이상으로 예측될 때 자동으로 알림이 발송되도록 해야 하며 모델의 성능을 주기적으로 모니터링하고 필요시 재학습을 수행하는 유지관리 체계를 마련해야 한다.

마지막으로 현장 운영자의 도메인 지식과 AI 예측 결과를 결합한 의사결정 프로세스를 확립해야 한다. AI는 보조 도구이며, 최종 판단은 전문가의 경험과 지식을 바탕으로 이루어져야 한다.

9. 참고문헌

1. 김동일, 박진영, 이상호. (2020). 머신러닝 기법을 이용한 상수원 탁도 예측 모델 개발. 한국물환경학회지, 36(4), 342-351.
2. 박상진. (2023). 시계열 및 인공지능 예측. 강의 교안, 한양대학교.
3. 이재수, 김민지. (2019). LSTM 을 활용한 수질 예측 시스템 구축 사례 연구. 한국수자원학회논문집, 52(3), 187-196.
4. 정우성, 최현일. (2021). XGBoost 와 LightGBM 을 이용한 정수장 탁도 예측. 대한환경공학회지, 43(5), 412-421.
5. 한국수자원공사. (2022). 상수원 수질 관리 및 예측 시스템 구축 보고서.
6. Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13(1), 281-305.
7. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.
8. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785-794.
9. Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078.
10. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735-1780.
11. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems, 30, 3146-3154.

10. 부록

10.1 Appendix

본 연구에서 사용된 모든 분석 코드는 R 과 Python 언어로 구현되었다. R 은 단변량 시계열 모델 및 다변량 머신러닝 모델 구축에 사용되었으며, Python 은 딥러닝 모델 (LSTM, GRU) 및 통합 분석에 활용되었다. 본 부록에서는 재현 가능성(Reproducibility)을 확보하기 위해 핵심 코드를 발췌하여 제시하며, 전체 코드는 별첨 파일로 제공한다.

A. R 코드 (단변량 시계열 모델 및 다변량 머신러닝)

A.1 데이터 전처리 및 변수 선정

[1] 변동계수(CV) 기반 1 차 필터링

변동계수 계산 및 필터링

```
cv_threshold <- 5
```

```
for(col in names(data)) {  
  if(is.numeric(data[[col]])) {  
    cv <- (sd(data[[col]], na.rm = TRUE) / mean(data[[col]], na.rm = TRUE)) * 100  
  
    if(cv < cv_threshold) {  
      cat(sprintf("제거 변수: %s (CV = %.2f%%)\n", col, cv))  
      data[[col]] <- NULL  
    }  
  }  
}
```

결과: up_pH (CV=4.34%), up_water_level (CV=0.47%) 제거

[2] LASSO 및 Elastic Net 회귀

정규화 회귀(Regularized Regression)를 통해 중요 변수를 선별하였다.

```
library(glmnet)
```

```
# Train/Test 데이터 분할 (80:20)
```

```
train_ratio <- 0.8
```

```

train_size <- floor(nrow(data) * train_ratio)
train_data <- data[1:train_size, ]
test_data <- data[(train_size + 1):nrow(data), ]

# 독립변수와 종속변수 분리
X_train <- as.matrix(train_data %>% select(where(is.numeric), -turbidity))
y_train <- train_data$turbidity

# LASSO 회귀 (alpha = 1.0)
lasso_model <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 5)
lasso_coef <- coef(lasso_model, s = "lambda.min")
lasso_selected <- rownames(lasso_coef)[which(lasso_coef != 0)][-1]

cat("LASSO 선택 변수:", length(lasso_selected), "개\n")
print(lasso_selected)

# Elastic Net 회귀 (alpha = 0.5)
enet_model <- cv.glmnet(X_train, y_train, alpha = 0.5, nfolds = 5)
enet_coef <- coef(enet_model, s = "lambda.min")
enet_selected <- rownames(enet_coef)[which(enet_coef != 0)][-1]

cat("Elastic Net 선택 변수:", length(enet_selected), "개\n")

```

[3] VIF(Variance Inflation Factor) 검증

```

library(car)

# 최종 후보 변수로 회귀 모델 구축
formula_str <- paste("turbidity ~", paste(candidate_vars, collapse = " + "))
lm_model <- lm(as.formula(formula_str), data = train_data)

# VIF 계산
vif_values <- vif(lm_model)
print(vif_values)

# VIF > 10 인 변수 제거
high_vif_vars <- names(vif_values[vif_values > 10])
cat("VIF > 10 인 변수:", high_vif_vars, "\n")

final_selected <- setdiff(candidate_vars, high_vif_vars)
cat("최종 선정 변수:", length(final_selected), "개\n")
print(final_selected)

# 결과: 19 개 변수 선정 (alkalinity, blue_green_algae, diatomeae, EC, pH,

```

```
# TOC, up_EC, up_pH, up_TOC, up_turbidity, algae, blue_algae, Geosmin,  
# MIB_2, up_TN, wind_velocity, cryptophyceae, temp, up_TP)
```

A.2 ARIMA 모델 구축 및 진단

[1] 시계열 정상성 검정

```
library(tseries)
```

원 데이터 ADF 검정

```
turbidity_ts <- ts(train_data$turbidity, frequency = 24)  
adf_original <- adf.test(turbidity_ts, alternative = "stationary")
```

```
cat("ADF 검정 통계량:", adf_original$statistic, "\n")  
cat("p-value:", adf_original$p-value, "\n")
```

```
if(adf_original$p.value < 0.05) {  
  cat("결론: 원 데이터가 정상성을 만족합니다 ( $p < 0.05$ )\n")  
} else {  
  cat("결론: 비정상 시계열입니다. 차분이 필요합니다.\n")  
}
```

1 차 차분 후 재검정

```
turbidity_diff1 <- diff(turbidity_ts, differences = 1)  
adf_diff1 <- adf.test(turbidity_diff1, alternative = "stationary")
```

```
cat("\n1 차 차분 후 ADF p-value:", adf_diff1$p.value, "\n")
```

ACF/PACF 시각화

```
par(mfrow = c(2, 2))  
acf(turbidity_ts, lag.max = 100, main = "원 데이터 ACF")  
pacf(turbidity_ts, lag.max = 100, main = "원 데이터 PACF")  
acf(turbidity_diff1, lag.max = 100, main = "1 차 차분 후 ACF")  
pacf(turbidity_diff1, lag.max = 100, main = "1 차 차분 후 PACF")  
par(mfrow = c(1, 1))
```

[2] 자동 ARIMA 차수 선정

```
library(forecast)
```

```
# auto.arima 를 통한 최적 차수 자동 선정
arima_model <- auto.arima(turbidity_ts,
                          stepwise = TRUE,
                          approximation = TRUE,
                          seasonal = TRUE)
```

```
# 모델 요약
summary(arima_model)
```

```
# 선정된 모델: ARIMA(2,1,3)(1,0,0)[24]
```

```
# - AR 차수 (p): 2
```

```
# - 차분 차수 (d): 1
```

```
# - MA 차수 (q): 3
```

```
# - 계절 AR 차수 (P): 1, 주기: 24 시간
```

```
# 모델 진단
tsdiag(arima_model)
```

[3] 예측 및 성능 평가

```
# Test 데이터 예측
h_forecast <- nrow(test_data)
arima_pred <- forecast(arima_model, h = h_forecast)
```

```
# 성능 지표 계산
actual_test <- test_data$turbidity
mse_arima <- mean((actual_test - arima_pred$mean)^2)
mae_arima <- mean(abs(actual_test - arima_pred$mean))
rmse_arima <- sqrt(mse_arima)
```

```
cat("ARIMA 모델 성능:\n")
cat(" MSE:", round(mse_arima, 4), "\n")
cat(" MAE:", round(mae_arima, 4), "\n")
cat(" RMSE:", round(rmse_arima, 4), "\n")
```

```
# 예측 결과 시각화
plot(test_data$measure_date, actual_test, type = "l",
     main = "ARIMA 예측 결과", xlab = "날짜", ylab = "탁도 (NTU)",
     col = "black", lwd = 2)
lines(test_data$measure_date, arima_pred$mean, col = "red", lwd = 2, lty = 2)
legend("topright", legend = c("실제", "예측"),
     col = c("black", "red"), lty = c(1, 2), lwd = 2)
```

A.3 Random Forest 모델

```
library(randomForest)

# 최종 선정된 19 개 변수로 데이터 준비
X_train_selected <- as.matrix(train_data[, final_selected])
X_test_selected <- as.matrix(test_data[, final_selected])

# Random Forest 모델 학습
rf_model <- randomForest(
  x = X_train_selected,
  y = y_train,
  ntree = 100,      # 트리 개수
  mtry = 6,         # 각 분할에서 고려할 변수 개수
  importance = TRUE, # 변수 중요도 계산
  nodesize = 5,     # 최소 노드 크기
  maxnodes = NULL   # 최대 노드 수 제한 없음
)

# 모델 요약
print(rf_model)

# 예측
rf_pred <- predict(rf_model, X_test_selected)

# 성능 평가
mse_rf <- mean((actual_test - rf_pred)^2)
mae_rf <- mean(abs(actual_test - rf_pred))
r2_rf <- 1 - sum((actual_test - rf_pred)^2) / sum((actual_test - mean(actual_test))^2)

cat("\nRandom Forest 성능:\n")
cat(" MSE:", round(mse_rf, 4), "\n")
cat(" MAE:", round(mae_rf, 4), "\n")
cat(" R²:", round(r2_rf, 4), "\n")

# 변수 중요도 시각화
varImpPlot(rf_model, main = "Random Forest 변수 중요도")

# 상위 10 개 중요 변수 출력
importance_scores <- importance(rf_model)
top10_vars <- head(importance_scores[order(importance_scores[, 1],
```

```
decreasing = TRUE), ], 10)  
print(top10_vars)
```

B. Python 코드 (딥러닝 모델: LSTM 및 GRU)

B.1 데이터 전처리 및 시퀀스 생성

```
import numpy as np  
import pandas as pd  
from sklearn.preprocessing import MinMaxScaler  
from tensorflow import keras  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import LSTM, GRU, Dense, Dropout  
from tensorflow.keras.callbacks import EarlyStopping  
  
# 데이터 로드 (CSV 파일)  
df = pd.read_csv('상수원_통합_수질_시계열_데이터.csv', encoding='euc-kr')  
df['measure_date'] = pd.to_datetime(df['measure_date'])  
df = df.sort_values('measure_date').reset_index(drop=True)  
  
# Train/Test 분할 (80:20)  
split_idx = int(len(df) * 0.8)  
train_df = df.iloc[:split_idx].copy()  
test_df = df.iloc[split_idx:].copy()  
  
# 타겟 변수 및 입력 변수 설정  
target = 'turbidity'  
feature_cols = ['alkalinity', 'blue_green_algae', 'diatomeae', 'EC', 'pH',  
                'TOC', 'up_EC', 'up_pH', 'up_TOC', 'up_turbidity', 'algae',  
                'blue_algae', 'Geosmin', 'MIB_2', 'up_T-N', 'wind_velocity',  
                'cryptophyceae', 'temp', 'up_T-P']  
  
# 데이터 추출  
X_train = train_df[feature_cols].values  
y_train = train_df[target].values  
X_test = test_df[feature_cols].values  
y_test = test_df[target].values  
  
print(f"X_train shape: {X_train.shape}")  
print(f"y_train shape: {y_train.shape}")
```

B.2 데이터 스케일링

LSTM 과 GRU 는 입력값의 스케일에 민감하므로 MinMaxScaler 를 사용하여

0~1 범위로 정규화하였다.

X, y 모두 스케일링

```
scaler_X = MinMaxScaler()
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

scaler_y = MinMaxScaler()
y_train_scaled = scaler_y.fit_transform(y_train.reshape(-1, 1)).flatten()
y_test_scaled = scaler_y.transform(y_test.reshape(-1, 1)).flatten()
```

```
print("스케일링 완료: X, y 모두 0~1 범위로 변환")
```

B.3 시퀀스 데이터 생성 함수

LSTM 과 GRU 는 시계열 데이터를 3 차원 형태 (samples, time_steps, features)로 입력받아야 한다. 본 연구에서는 과거 24 시간의 데이터를 활용하여 현재 시점의 탁도를 예측하도록 설계하였다.

```
def create_sequences(X, y, time_steps=24):
    """
    시계열 데이터를 LSTM/GRU 입력용 시퀀스로 변환

    Parameters:
    X : array-like, shape (n_samples, n_features)
        입력 특성 데이터 (스케일링 완료)
    y : array-like, shape (n_samples,)
        타겟 변수 (스케일링 완료)
    time_steps : int, default=24
        시퀀스 길이 (과거 몇 시간의 데이터를 사용할지)

    Returns:
    X_seq : array, shape (n_samples-time_steps, time_steps, n_features)
        시퀀스 형태로 변환된 입력 데이터
    y_seq : array, shape (n_samples-time_steps,)
        시퀀스에 대응하는 타겟 변수

    Example:
    과거 24 시간의 데이터로 25 번째 시간의 탁도 예측
    X_seq[0] = [X[0], X[1], ..., X[23]] → y_seq[0] = y[24]
    X_seq[1] = [X[1], X[2], ..., X[24]] → y_seq[1] = y[25]
```



```

"""
Xs, ys = [], []

for i in range(len(X) - time_steps):
    # 과거 time_steps 시간의 데이터를 하나의 시퀀스로
    Xs.append(X[i:i+time_steps])
    # time_steps 이후의 값을 예측 대상으로
    ys.append(y[i+time_steps])

return np.array(Xs), np.array(ys)

# 시퀀스 데이터 생성 (24 시간 window)
time_steps = 24

X_train_seq, y_train_seq = create_sequences(X_train_scaled, y_train_scaled,
                                             time_steps)
X_test_seq, y_test_seq = create_sequences(X_test_scaled, y_test_scaled,
                                           time_steps)

print(f" X_train_seq shape: {X_train_seq.shape}")
print(f" y_train_seq shape: {y_train_seq.shape}")
print(f" X_test_seq shape: {X_test_seq.shape}")
print(f" y_test_seq shape: {y_test_seq.shape}")

# 예시:
# X_train_seq.shape = (21835, 24, 19)
# → 21,835 개 샘플, 각 샘플은 24 시간의 과거 데이터, 19 개 변수

```

B.4 LSTM 모델 구축

[1] 모델 아키텍처 설계

```

def build_lstm_model(input_shape):
    """
    LSTM 기반 택도 예측 모델

    Architecture:

    1. LSTM Layer 1: 64 units, return_sequences=True
       - 시퀀스의 모든 시점 출력을 다음 층으로 전달
       - 활성화 함수: tanh (LSTM 기본값)

    2. Dropout 1: 0.3
    
```

- 과적합 방지를 위한 30% 뉴런 무작위 제거

3. LSTM Layer 2: 32 units, return_sequences=False

- 시퀀스의 마지막 출력만 다음 층으로 전달

4. Dropout 2: 0.2

5. Dense Layer: 16 units, ReLU activation

- 비선형 변환을 위한 완전 연결 층

6. Output Layer: 1 unit (타도 예측값)

Parameters:

input_shape : tuple

(time_steps, n_features) 형태

예: (24, 19) → 24 시간, 19 개 변수

Returns:

model : Keras Sequential model

"""

```
model = Sequential([
```

```
    # 첫 번째 LSTM 층 (64 units)
```

```
    LSTM(64,
```

```
        activation='tanh',
```

```
        return_sequences=True, # 다음 LSTM 층으로 시퀀스 전달
```

```
        input_shape=input_shape),
```

```
    # Dropout 으로 과적합 방지
```

```
    Dropout(0.3),
```

```
    # 두 번째 LSTM 층 (32 units)
```

```
    LSTM(32,
```

```
        activation='tanh',
```

```
        return_sequences=False), # 마지막 출력만 전달
```

```
    Dropout(0.2),
```

```
    # 완전 연결 층 (Dense layer)
```

```
    Dense(16, activation='relu'),
```

```
    # 출력 층 (1 개 뉴런 = 타도 예측값)
```

```
    Dense(1)
```

```
])
```

```

# 모델 컴파일
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='mse',    # 평균 제곱 오차 (회귀 문제)
    metrics=['mae'] # 평균 절대 오차 (모니터링용)
)

return model

```

[2] 모델 학습

```

# 입력 형태 정의
input_shape = (X_train_seq.shape[1], X_train_seq.shape[2])
# input_shape = (24, 19)

# Early Stopping 콜백 설정
early_stop = EarlyStopping(
    monitor='val_loss',    # 검증 손실을 모니터링
    patience=20,           # 20 에포크 동안 개선 없으면 중단
    restore_best_weights=True, # 최적 가중치로 복원
    verbose=1
)

# LSTM 모델 생성
lstm_model = build_lstm_model(input_shape)

# 모델 구조 출력
lstm_model.summary()

# 모델 학습
history_lstm = lstm_model.fit(
    X_train_seq, y_train_seq,
    epochs=100,        # 최대 100 에포크
    batch_size=32,     # 배치 크기
    validation_split=0.2, # Train 의 20%를 검증용으로
    callbacks=[early_stop],
    verbose=1
)

# LSTM 학습 완료

```

```

print(f" 최종 에포크: {len(history_lstm.history['loss'])}")
print(f" 최종 Train Loss: {history_lstm.history['loss'][-1]:.4f}")
print(f" 최종 Val Loss: {history_lstm.history['val_loss'][-1]:.4f}")

```

[3] 예측 및 역변환

```

# Test 데이터 예측 (스케일링된 값)
lstm_test_pred_scaled = lstm_model.predict(X_test_seq, verbose=0).flatten()

# 원래 스케일로 역변환
lstm_test_pred = scaler_y.inverse_transform(
    lstm_test_pred_scaled.reshape(-1, 1)
).flatten()

# 실제 y_test 도 역변환 (비교를 위해)
y_test_actual = scaler_y.inverse_transform(
    y_test_seq.reshape(-1, 1)
).flatten()

print(f"\n 예측 완료:")
print(f" 예측값 shape: {lstm_test_pred.shape}")
print(f" 예측값 범위: {lstm_test_pred.min():.2f} ~ {lstm_test_pred.max():.2f} NTU")

```

[4] 성능 평가

```

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# 성능 지표 계산
mse_lstm = mean_squared_error(y_test_actual, lstm_test_pred)
mae_lstm = mean_absolute_error(y_test_actual, lstm_test_pred)
rmse_lstm = np.sqrt(mse_lstm)
r2_lstm = r2_score(y_test_actual, lstm_test_pred)

print(f"\nLSTM 모델 성능:")
print(f" MSE: {mse_lstm:.4f}")
print(f" MAE: {mae_lstm:.4f}")
print(f" RMSE: {rmse_lstm:.4f}")
print(f" R²: {r2_lstm:.4f}")

```

B.5 GRU 모델 구축

GRU(Gated Recurrent Unit)는 LSTM 의 간소화 버전으로, 게이트가 2 개(Reset, Update)만 있어 계산 효율성이 높으면서도 유사한 성능을 보인다.

```
def build_gru_model(input_shape):
```

```
    """
```

```
    GRU 기반 탁도 예측 모델
```

```
    Architecture:
```

```
    LSTM 과 동일한 구조이나 LSTM 층을 GRU 층으로 대체
```

- GRU 는 LSTM 보다 파라미터가 적어 학습 속도가 빠름
- 단기 의존성이 강한 시계열에서 효과적

```
    """
```

```
    model = Sequential([
```

```
        # 첫 번째 GRU 층 (64 units)
```

```
        GRU(64,  
            activation='tanh',  
            return_sequences=True,  
            input_shape=input_shape),
```

```
        Dropout(0.3),
```

```
        # 두 번째 GRU 층 (32 units)
```

```
        GRU(32,  
            activation='tanh',  
            return_sequences=False),
```

```
        Dropout(0.2),
```

```
        # 완전 연결 층
```

```
        Dense(16, activation='relu'),
```

```
        # 출력 층
```

```
        Dense(1)
```

```
    ])
```

```
    model.compile(
```

```
        optimizer=keras.optimizers.Adam(learning_rate=0.001),
```

```
        loss='mse',
```

```
        metrics=['mae']
```

```
    )
```

```
    return model
```

GRU 모델 생성 및 학습

```
gru_model = build_gru_model(input_shape)
gru_model.summary()
```

```
history_gru = gru_model.fit(
    X_train_seq, y_train_seq,
    epochs=100,
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)
```

예측

```
gru_test_pred_scaled = gru_model.predict(X_test_seq, verbose=0).flatten()
gru_test_pred = scaler_y.inverse_transform(
    gru_test_pred_scaled.reshape(-1, 1)
).flatten()
```

성능 평가

```
mse_gru = mean_squared_error(y_test_actual, gru_test_pred)
mae_gru = mean_absolute_error(y_test_actual, gru_test_pred)
rmse_gru = np.sqrt(mse_gru)
r2_gru = r2_score(y_test_actual, gru_test_pred)
```

```
print(f"\nGRU 모델 성능:")
print(f" MSE: {mse_gru:.4f}")
print(f" MAE: {mae_gru:.4f}")
print(f" RMSE: {rmse_gru:.4f}")
print(f" R²: {r2_gru:.4f}")
```

B.6 LSTM vs GRU 비교

성능 비교 표

```
import pandas as pd
```

```
comparison = pd.DataFrame({
    'Model': ['LSTM', 'GRU'],
    'MSE': [mse_lstm, mse_gru],
    'MAE': [mae_lstm, mae_gru],
    'RMSE': [rmse_lstm, rmse_gru],
    'R²': [r2_lstm, r2_gru],
    'Parameters': [lstm_model.count_params(), gru_model.count_params()],
})
```

```

        'Training Time (epochs)': [len(history_lstm.history['loss']),
                                   len(history_gru.history['loss'])]
    })

# LSTM vs GRU 성능 비교
print(comparison.to_string(index=False))

# 시각화
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(15, 5))

# 학습 곡선 비교
axes[0].plot(history_lstm.history['loss'], label='LSTM Train Loss', color='blue')
axes[0].plot(history_lstm.history['val_loss'], label='LSTM Val Loss',
              color='blue', linestyle='--')
axes[0].plot(history_gru.history['loss'], label='GRU Train Loss', color='red')
axes[0].plot(history_gru.history['val_loss'], label='GRU Val Loss',
              color='red', linestyle='--')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss (MSE)')
axes[0].set_title('Training History: LSTM vs GRU')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 예측 vs 실제 비교
axes[1].scatter(y_test_actual, lstm_test_pred, alpha=0.5, label='LSTM', color='blue')
axes[1].scatter(y_test_actual, gru_test_pred, alpha=0.5, label='GRU', color='red')
axes[1].plot([y_test_actual.min(), y_test_actual.max()],
              [y_test_actual.min(), y_test_actual.max()],
              'k--', lw=2, label='Perfect Prediction')
axes[1].set_xlabel('Actual Turbidity (NTU)')
axes[1].set_ylabel('Predicted Turbidity (NTU)')
axes[1].set_title('Actual vs Predicted')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

10.2 요약

상수원 탁도는 정수처리 공정의 안정성과 수돗물 안전성을 결정하는 핵심 수질 지표이다.

집중호우 시 탁도는 평상시 1~5 NTU 에서 수백 NTU 까지 급등하여 응집제 사용량 15% 증가,

처리 효율 20% 지연 등 심각한 운영 문제를 야기한다. 기존 연구는 주로 상류 중심 예측에 치중되어 실제 취수 지점의 복합적 상황을 반영하지 못하는 한계가 있었다.

본 연구는 실제 정수장이 위치한 하류 상수원의 탁도를 예측하기 위해 2019 년 1 월부터 2022 년 6 월까지 3.5 년간 1 시간 단위로 수집된 30,642 개 관측치를 활용하였다. 수질, 수문, 기상, 생물학적 인자를 포함한 28 개 변수를 변동계수, LASSO, Elastic Net, VIF 등 체계적 방법으로 19 개 핵심 변수로 선정하였다. 단변량 시계열 모델(Naive, Mean, SES, Holt, Holt-Winters, ARIMA), 다변량 머신러닝 모델(Random Forest, XGBoost, LightGBM, SVM), 딥러닝 모델(LSTM, GRU) 등 총 11 개 모델을 구축하고 Train/Test(80:20) 분할을 통해 성능을 비교하였다.

분석 결과, Random Forest 모델이 MSE 1.12, MAE 0.88, R^2 0.85 로 가장 우수한 성능을 보였으며, 상류 탁도(up_turbidity)가 변수 중요도 1 위로 확인되었다. 조류(algae), 수온, 방류량 등 환경 변수도 유의미한 영향을 미쳤다. 단순 평균 모델이 MSE 1.03 으로 최저값을 기록했으나 R^2 -0.04 로 실제 변동 추적력이 없어 실용성이 떨어졌다. ARIMA(2,1,3)(1,0,0)[24] 모델은 24 시간 계절성을 반영하였으나 MSE 1.77 로 급변 상황 대응력이 부족했다. LSTM 과 GRU 는 시계열 특성을 학습하였으나 데이터량 부족과 하이퍼파라미터 최적화 미흡으로 Random Forest 에 미달하였다.

본 연구는 하류 상수원 타겟팅, 다변량 통합, 해석 가능한 모델 제시를 통해 기존 연구의 한계를 극복하였으며, 정수장 운영 최적화와 응집제 사용량 사전 조절에 실질적으로 기여할 수 있음을 입증하였다. 다만 테스트 기간의 평온성으로 극단적 고탁도 이벤트 예측력 검증이 부족하고, 강우량 데이터 미포함이 한계로 남아있다. 향후 강우량 추가, 장기 예측 확장, 실시간 스트리밍 시스템 구축이 필요하다.

10.3 팀원별 업무 분담

업무 영역	담당자	세부 내역
-------	-----	-------

1. R 코딩	정예지	• 데이터 전처리 • 변수 선정 (LASSO, Elastic Net, VIF 등) • 단변량 시계열 모델링 (Naive, Mean, SES, Holt, Holt-Winters, ARIMA) • 다변량 머신러닝 (Random Forest, XGBoost, LightGBM, SVM) • R 시각화 (상관관계 히트맵, 변수 중요도 등)
2. Python 코딩	김성현	• Lag Feature 생성 • 다중공선성 제거 • 머신러닝 모델 (Linear Regression, Decision Tree, LightGBM, MLP) • 딥러닝 모델 (LSTM, GRU) • 시퀀스 데이터 생성 • Python 시각화 (시계열 그래프, 성능 비교 차트)
3. 자료 조사	조강민 (보조: 김성현 정예지)	• 문헌 조사 (국내외 논문 10 편 이상) • 선행 연구 분석 • 기존 연구의 한계점 정리 • 본 연구의 차별성 도출 • 참고문헌 목록 작성 • 인용 형식 검토
4. 과제 작성	정예지	• Abstract 작성 • 연구 배경 및 필요성 • 연구 목적 및 가설 • 데이터 수집 및 분석 • 변수 선정 과정 설명 • 기초 통계 분석 • 모델 구축 및 결과 • 연구 결과 해석 및 시사점 • 한계 및 결론 • 전체 보고서 편집

5. 검토	김성헌 조강민	• 전체 보고서 내용 검토 • 오류 및 오타자 수정 • 논리적 흐름 점검 • 그래프 및 표 완성도 확인 • 평가 기준 충족 여부 점검 • 최종 포맷팅
6. 발표	김성헌 조강민	• 발표 대본 작성 • 발표 시연 • 질의응답 준비